

Learning to efficiently analyse large adaptation spaces of Internet of Things applications

Federico Quin

Thesis voorgedragen tot het behalen
van de graad van Master of Science
in de ingenieurswetenschappen:
computerwetenschappen, hoofdoptie
Software engineering

Promotoren:

Prof. dr. D. Weyns
Dr. S. Michiels

Assessoren:

Prof. dr. ir. Y. Berbers
Prof. dr. ir. D. Nuyens

Begeleider:

Prof. dr. D. Weyns

© Copyright KU Leuven

Without written permission of the thesis supervisors and the author it is forbidden to reproduce or adapt in any form or by any means any part of this publication. Requests for obtaining the right to reproduce or utilize parts of this publication should be addressed to the Departement Computerwetenschappen, Celestijnenlaan 200A bus 2402, B-3001 Heverlee, +32-16-327700 or by email info@cs.kuleuven.be.

A written permission of the thesis supervisors is also required to use the methods, products, schematics and programmes described in this work for industrial or commercial use, and for submitting this publication in scientific contests.

Zonder voorafgaande schriftelijke toestemming van zowel de promotoren als de auteur is overnemen, kopiëren, gebruiken of realiseren van deze uitgave of gedeelten ervan verboden. Voor aanvragen tot of informatie i.v.m. het overnemen en/of gebruik en/of realisatie van gedeelten uit deze publicatie, wend u tot het Departement Computerwetenschappen, Celestijnenlaan 200A bus 2402, B-3001 Heverlee, +32-16-327700 of via e-mail info@cs.kuleuven.be.

Voorafgaande schriftelijke toestemming van de promotoren is eveneens vereist voor het aanwenden van de in deze masterproef beschreven (originele) methoden, producten, schakelingen en programma's voor industrieel of commercieel nut en voor de inzending van deze publicatie ter deelname aan wetenschappelijke prijzen of wedstrijden.

Preface

Firstly I would like to sincerely thank my supervisor Danny Weyns. His guidance and feedback were invaluable in the making of this thesis. When in doubt, I could always reach out with any question I had. Appointments for a quick followup or extended discussion were always quickly made. His expertise and accommodating attitude were a crucial part in bringing this research to a successful conclusion.

I would also like to thank my family at home for supporting me throughout the year. Even though some periods were more stressful than others, they were always there to encourage and help me in any way they could. Thank you very much for all the help.

Lastly, I would like to express my gratitude to following the people I have collaborated with in this research: Sarpreet Singh Buttar, Sam Michiels, Usman Iftikhar and Thomas Bamelis.

Federico Quin

Abstract

When a self-adaptive system detects that its adaptation goals may be compromised, it needs to determine how to adapt to ensure its goals. To that end, the system can analyze the possible options for adaptation, i.e., the adaptation space, and pick the best option that achieves the goals. Such analysis can be resource and time consuming, in particular when rigorous analysis methods are applied. Hence, exhaustively analyzing all options may be infeasible for systems with large adaptation spaces. This problem is further complicated as the adaptation options typically include uncertainty parameters that can only be resolved at runtime. In this research, we present a machine learning approach to tackle this problem. This approach enhances the traditional MAPE-K feedback loop with a learning module that selects subsets of adaptation options from a large adaptation space to support the analyzer with performing efficient analysis. We instantiate the approach for two concrete learning techniques, classification and regression, and evaluate the approaches for two instances of an Internet of Things application for smart environment monitoring with different sizes of adaptation spaces. The evaluation shows that both learning approaches reduce the adaptation space significantly without noticeable effect on realizing the adaptation goals.

Contents

Preface	i
Abstract	ii
Samenvatting	iii
List of Figures and Tables	vi
1 Introduction	1
2 Self-adaptation	3
2.1 General introduction	3
2.2 DeltaIoT	7
2.3 Quality models	12
2.4 Conclusion	16
3 Methodology	17
3.1 Problem in detail	17
3.2 Architecture	19
3.3 Concrete application to DeltaIoT	20
3.4 Conclusion	23
4 Implementation	25
4.1 DeltaIoT simulator	25
4.2 Offline activities	26
4.3 Online activities	30
4.4 Conclusion	35
5 Evaluation	37
5.1 Setup machine learning module	37
5.2 Comparison exhaustive approach	43
5.3 Threats to validity	48
5.4 Conclusion	49
6 Related work	51
6.1 Learning in self-adaptive systems	51
6.2 Analyzer optimizations	52
7 Conclusion	55
7.1 Future work	56
7.2 Reflection	56

A Thesis poster	59
B Scientific paper (SEAMS)	61
C Dutch summary scientific paper	75
Bibliography	81

List of Figures and Tables

List of Figures

2.1	Interaction between managed and managing system	4
2.2	DeltaIoT.v1 deployment at KU Leuven Campus [1]	7
2.3	Physical layout of the DeltaIoT.v2 network	8
2.4	Network layout of the DeltaIoT.v2 network	9
2.5	Adaptation space in DeltaIoT.v2 for a single network cycle	10
2.6	Influence of uncertainties on qualities of a single configuration	11
2.7	Example configuration of two links in the DeltaIoT.v2 network	12
2.8	Formal model used to calculate the packet loss of the network	13
2.9	Formal model used to calculate the latency of the network	14
2.10	Formal model used to calculate the energy consumption of the network	15
3.1	Original architecture and workflow of the analyzer	19
3.2	Architecture and workflow of the analyzer extended with a machine learning module	20
3.3	Architecture and workflow of the analyzer, filled in with a scaler and classifier	22
3.4	Architecture and workflow of the analyzer, filled in with a scaler and regressor	23
4.1	Diagram of the different offline steps, concretely for classification	29
4.2	Diagram of the different steps in the testing and training cycles concretely for classification	33
5.1	The importance scores of all the different features in the DeltaIoT.v2 network	38
5.2	Overall results for DeltaIoT.v1	44
5.3	Selected and explored adaptation options in a single cycle (DeltaIoT.v1).	44
5.4	Overall results for DeltaIoT.v2	45
5.5	Selected and explored adaptation options in a single cycle using classification (DeltaIoT.v2)	46
5.6	Selected and explored adaptation options in a single cycle using regression (DeltaIoT.v2)	46
5.7	Single configurations tracked through time (DeltaIoT.v2 with classification)	47

5.8	Single configurations tracked through time (DeltaIoT.v2 with classification)	48
-----	--	----

List of Tables

4.1	Example format of some collected offline data	27
4.2	Example of performing feature selection on the data from table 4.1 . . .	28
4.3	Example of performing model selection	29
4.4	Example of the different phases in a testing cycle	34
5.1	Chosen model combinations for classification and regression in both networks.	42
5.2	Metrics for the classifiers and regressors of DeltaIoT.v1 and DeltaIoT.v2	42

Chapter 1

Introduction

Modern software systems are expected to deal with uncertain operating conditions: e.g. changing network conditions and dynamic traffic in Internet of Things applications. If these conditions are not dealt with properly, system qualities could be compromised. To that end, self-adaptation provides a way for the system to deal with these uncertain conditions. The system is traditionally equipped with a feedback loop which keeps track of the system state and manages the system. When a system goal is compromised, the feedback loop is able to adjust the system configuration.

Verifying the system's qualities is usually done by using formal models and formal verification. With the increasing complexity of current software systems, the amount of possible system configurations which need to be formally verified grows larger and larger. Formal verification of those larger spaces might not be feasible anymore due to the verification process being resource and time consuming. This brings us to the following research question which we tackle in this thesis:

How can machine learning techniques be used to determine subsets of adaptation options from large adaptation spaces allowing a self-adaptive system to perform more efficient analysis without compromising the realization of the adaptation goals?

At first, an introduction about self-adaptation, and specifically the IoT applications which we use for evaluation, is given in chapter 2. Basic concepts such as feedback loops, uncertainties, quality models and adaptation in general are explained. The different characteristics and setup of the IoT networks are also discussed.

In chapter 3 we give a more in depth description of the problem statement and present a methodology to tackle this problem. The methodology is split into two parts: an abstract reusable component/module that is added to a well known feedback system, and the instantiation of the module with two well known machine learning techniques.

Chapter 4 goes through the implementation of the networks (a simulator that mimics the networks is used) and goes more in depth into the working of the machine learning module. The chapter introduces a full description of the workflow needed in the module (with a distinction between steps before and after the system is operational).

Once the module is proposed and the workflow of the module is described in detail, chapter 5 evaluates the results retrieved from applying this approach to both IoT networks. It discusses the strengths and drawbacks of the approach as well as threats to validity.

Chapter 6 gives an overview of the related work. It discusses other approaches to tackle the problem statement, as well as giving a general overview of how machine learning techniques are currently being used in self-adaptive systems.

Lastly chapter 7 gives a quick recap of the proposed approach in this thesis and the retrieved results from applying this approach to two case study IoT applications. We discuss a few possible directions for future work. Finally, I give a brief reflection on the past year about my experiences during this research.

We have, as part of this thesis, submitted a paper [2] to the international symposium on Software Engineering for Adaptive and self-Managing Systems (SEAMS). The paper was accepted and presented at the conference. The full paper can be found in appendix B.

Chapter 2

Self-adaptation

In this chapter we will firstly give an overview of what self-adaptation is, why self-adaptation is used and a brief explanation of the problem statement we tackle in this research. This problem statement and the proposed solution to address it are further described in chapter 3. Secondly we will give an overview of the systems on which we evaluate the proposed solution. The implementation of the solution and the analysis of the retrieved results are discussed in chapter 4 and chapter 5 respectively. Lastly, we discuss the formal quality models that we use for the internet of things networks to predict qualities at runtime for these systems.

2.1 General introduction

Self-adaptation is a software principle that supports systems which run in changing environments by adapting these systems so that the goals of the system remain satisfied. Adaptation is mainly done by adjusting the system configuration and/or system parameters. Self-adaptation is especially critical in applications that require business continuity and quality guarantees. An example of such a system is an IoT network: the system deals with changing network conditions (e.g. changing weather conditions) that have an effect on the qualities of the network (e.g. packet loss). If these uncertainties are not properly handled, the system qualities might be jeopardized [3–5]. Self-adaptation resolves these issues by equipping the system with a feedback mechanism. This mechanism can then trigger adjustments to the system configuration if necessary.

The application domain of self-adaptation is quite large. Examples of systems where self-adaptation proves useful are: Cyber physical systems, Personalized Web Tasking, Internet of Things networks, robotics, ... In such systems self-adaptation can be used in different forms. We follow the definition of four different types of self-adaptation in [9]: self-healing, self-configuration, self-optimization and self-protection. Self-healing concerns itself with detecting problems in a system, diagnosing it and repairing it. Self-configuration deals with the automatic configuration of systems. Self-optimization adjusts the system so that its performance/cost/... is optimized.

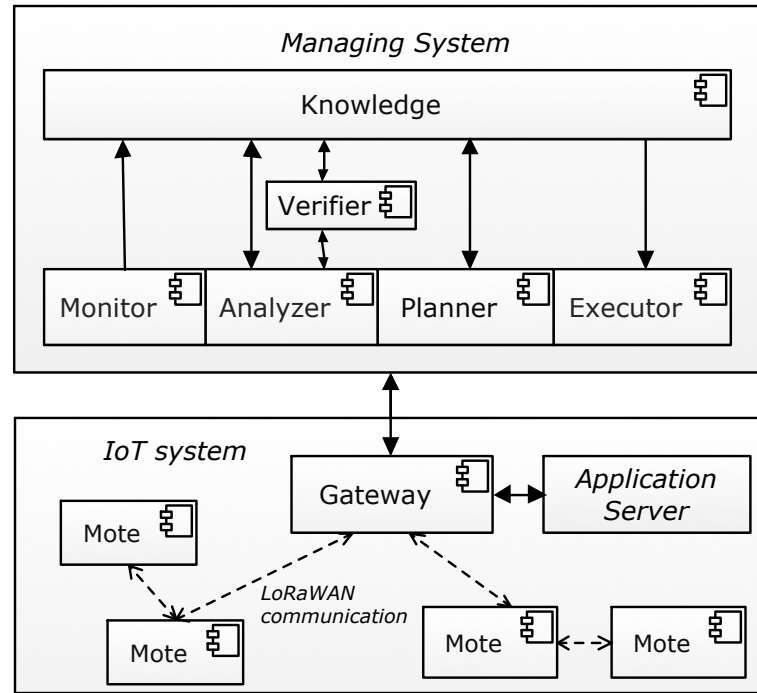


Figure 2.1: Interaction between managed and managing system

Lastly self-protection is concerned with the security of a system against malicious actors and/or failures. The goals and design of a self-adaptive system can vary depending on the type of self-adaptation that is required.

Self-adaptation is typically realized by means of an external feedback loop systems that is built on top of the existing system [6–9]. The feedback loop system is usually called the *managing system*, while the latter is called the *managed system*. The managing system keeps track of the conditions of the managed system at runtime. If it detects that the system goals are compromised, a plan has to be made in order to adjust the managed system and ensure the goals again. To that end, the possible options for adaptation, i.e., the possible system configurations, are analyzed. The best option of these analyzed options is selected to adapt the system. In this research, we refer to the set of possible options for adaptation as the *adaptation space*. Figure 2.1 gives an overview of how the managed and managing system interact (here, the managed system is the *IoT system*).

A well known feedback loop system is a MAPE-K loop [6, 10–13]. MAPE-K stands for Monitor, Analyzer, Planner, Executor and Knowledge. As the names suggest, the loop starts with monitoring the underlying managed system. This data is then stored in a shared knowledge component. The next step in the feedback loop is the analyzer phase. The analyzer evaluates the system configuration and the compliance of its

goals. If these goals are not met anymore, the analyzer starts to analyze the different adaptation options. After analyzing the options in the adaptation space, the planner evaluates the options in relation to the system goals. It chooses the best option according to these goals and composes an adaptation plan for the managed system. Finally the executor component goes through each step of the plan in order to adjust the configuration of the managed system to the newly selected adaptation option.

The analysis phase in this feedback loop is often aided by additional mechanisms. One of those mechanisms is the use of formal models at runtime to reason about properties of the different adaptation options, see e.g. [14–21]. In this research, we use the following definition from G. S. Blair et al. [22] to define a model at runtime:

“a model@run.time is a causally connected self-representation of the associated system that emphasizes the structure, behavior, or goals of the system from a problem space perspective [22].”

In other words: a model at runtime represents the underlying managed system in a specific way (e.g. modeling the behavior of the system). We design and use these models to deduct properties of this system when it is configured according to a chosen adaptation option. The details of how these models are designed and used is further discussed in section 2.3.

2.1.1 ActivFORMS

In this research we make use of ActivFORMS [23]. ActivFORMS stands for Active Formal Models for Self-Adaptation. It presents an approach on how to engineer a self-adaptive system while providing guarantees about the workings of this system. More concretely, the approach is designed with three goals in mind:

1. Reason about the correct behavior of the feedback loop system built for adaptation
2. Deal with dynamic adaptation goals and changes to verification models at runtime
3. Provide guarantees about the accuracy and confidence of the predictions made by the verification models

In this approach the feedback loop is initially modeled and formally verified. This allows us to reason about its correctness. Verification of the loop is typically done by checking a set of correctness properties about the feedback loop. Once the model is built and verified, a *Model Execution Engine* is used to execute the model that was designed in the previous step. Correctness about the feedback loop model is therefore preserved during later execution. The next step is using the model at runtime by

performing runtime verification and adapting the system if needed. The model execution engine is used to analyze different adaptation options [24]. The analyzer in ActivFORMS uses a statistical model checker as verifier [25, 26]. Depending on the adaptation goals of the system a decision is made to choose the optimal adaptation option. Lastly, ActivFORMS also offers the ability to change adaptation goals and the feedback loop model at runtime. The latter update triggers a trusted online update manager to make sure the correctness properties still hold for the new feedback loop model.

2.1.2 Uncertainties

A key aspect of self-adaptation is being aware and dealing with uncertainties in a system [5]. Perez-Palacin et al. [4] have defined a taxonomy on the different types of uncertainties that can be encountered. Three criteria are used to classify different type of uncertainties: location, level and nature.

The first criteria, location, reasons about the part of the model where the uncertainty is located. They make a distinction between *context uncertainty*, *model structural uncertainty* and *input parameters uncertainty*. Context uncertainty concerns itself with uncertainty about information of the system (e.g. information that was not included inside the model). Model structural uncertainty is concerned with the correctness of the model itself, in the sense of how accurately it models reality. Input parameters uncertainty describes the uncertainty of the values of the parameters used to evaluate the model.

The second criteria, level, is defined by the authors as follows:

"The level of uncertainty is where the uncertainty manifests itself along the spectrum between deterministic knowledge and total ignorance. [4]."

Five levels/orders are defined: from order 0 where there is no uncertainty up until order 4 where there is uncertainty about the order of uncertainty. In between we have orders that reason about being aware of uncertainties, lacking awareness of uncertainties and lacking process of becoming aware of uncertainties.

The final criteria, nature, is divided into two categories: *Epistemic* and *Aleatory*. Epistemic reasons about uncertainty being introduced as a result of lack/imperfection/... of the data. Aleatory are types of uncertainty introduced due to the nature of some processes where for example randomness in the process plays a part.

2.2 DeltaIoT

In this section, we will describe the *Internet of Things* networks on which the proposed methodology is evaluated. There are two networks which we use for evaluation: DeltaIoT.v1 and DeltaIoT.v2. Both networks share similar characteristics in terms of inner functioning. The first network is relatively small whilst the second network is larger.

2.2.1 DeltaIoT.v1

DeltaIoT.v1 consists of 15 Long-Range (LoRa) motes that are deployed at the Campus in Leuven as shown in figure 2.2. Each mote is equipped with a sensor that collects data that it relays to the gateway using wireless multi-hop communication. The network includes RFID sensors, passive infrared sensors, and heat sensors. The sensor data is forwarded to an end-user application to monitor the Campus area and take action when needed. Each mote is equipped with a buffer which keeps track of messages it receives from other motes. These buffered messages, and the messages generated by the mote itself, are then forwarded to the next hop in the network.



Figure 2.2: DeltaIoT.v1 deployment at KU Leuven Campus [1]

Communication is time-synchronized and organized in cycles with slots. Each slot defines a sender and a receiver mote. Motes are battery powered and sending messages dominates the energy cost. For our evaluation, we used a cycle time of 570 seconds (9.5 minutes), each cycle comprising slots that are allocated for communication between two motes. The slots of the first 8 minutes are used for data communication. The remaining slots are available to send messages for adapting the settings of motes.

2.2.2 DeltaIoT.v2

DeltaIoT.v2 consists of 37 motes and a central gateway which are deployed inside the computer science building of the Campus in Leuven. The layout of the motes and gateway can be found in figure 2.3. Similarly to DeltaIoT.v1, the motes are equipped with a sensor that collects data. This data is then forwarded to the central gateway. The motes also have a buffer to store messages it receives from other motes. As opposed to DeltaIoT.v1, where the network itself (links between motes and/or gateway) is fixed, DeltaIoT.v2 is deployed as a smart mesh. The links in the network are not fixed and can change at runtime.

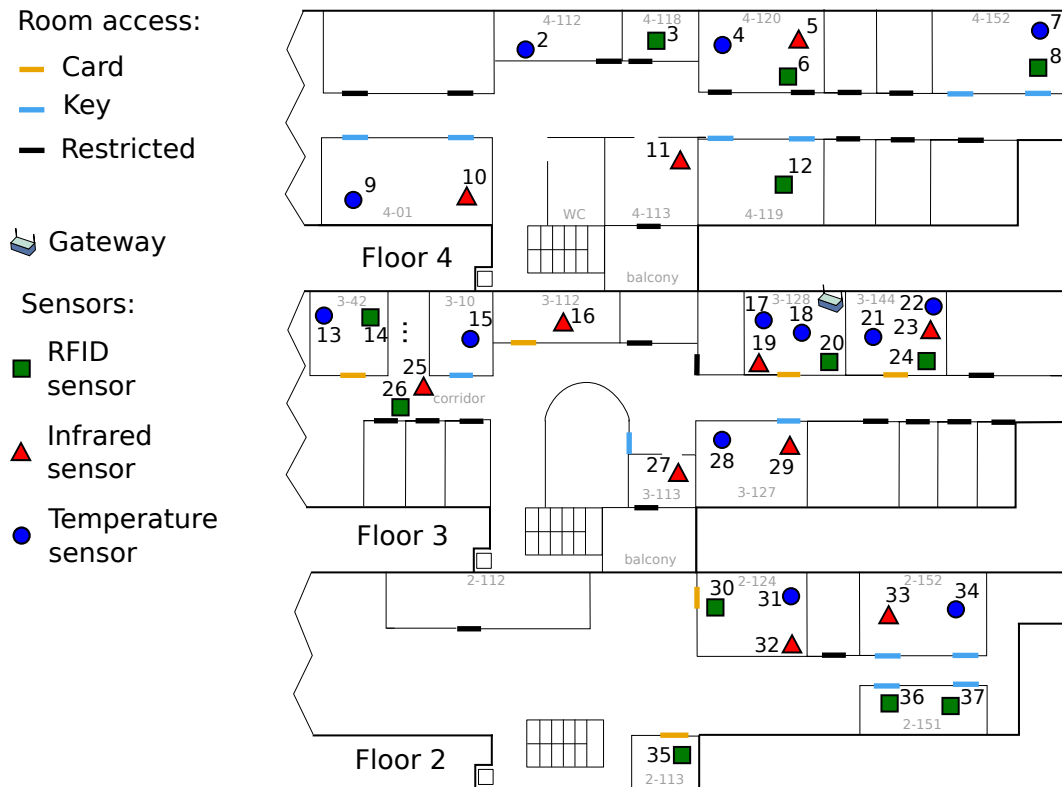


Figure 2.3: Physical layout of the DeltaIoT.v2 network

In this research we adjusted the network to conform with DeltaIoT.v1. Firstly we converted the mote types in DeltaIoT.v2 to the three types mentioned before: RFID sensors, passive infrared sensors and heat sensors. Secondly we adjusted the network to have fixed links. To do this, we manually constructed the links between motes and gateways. The choice of those fixed links were dependent on the proximity of the motes to each other (across floors and rooms). Figure 2.4 shows the resulting network layout of this procedure.

After converting the network to a fixed network layout, communication remains time-synchronized and organized in slots similar to DeltaIoT.v1. In our evaluation we use cycles that are configured to be 12 minutes long, 9.5 minutes for sending sensor data and 2.5 minutes for sending messages to adapt the motes.

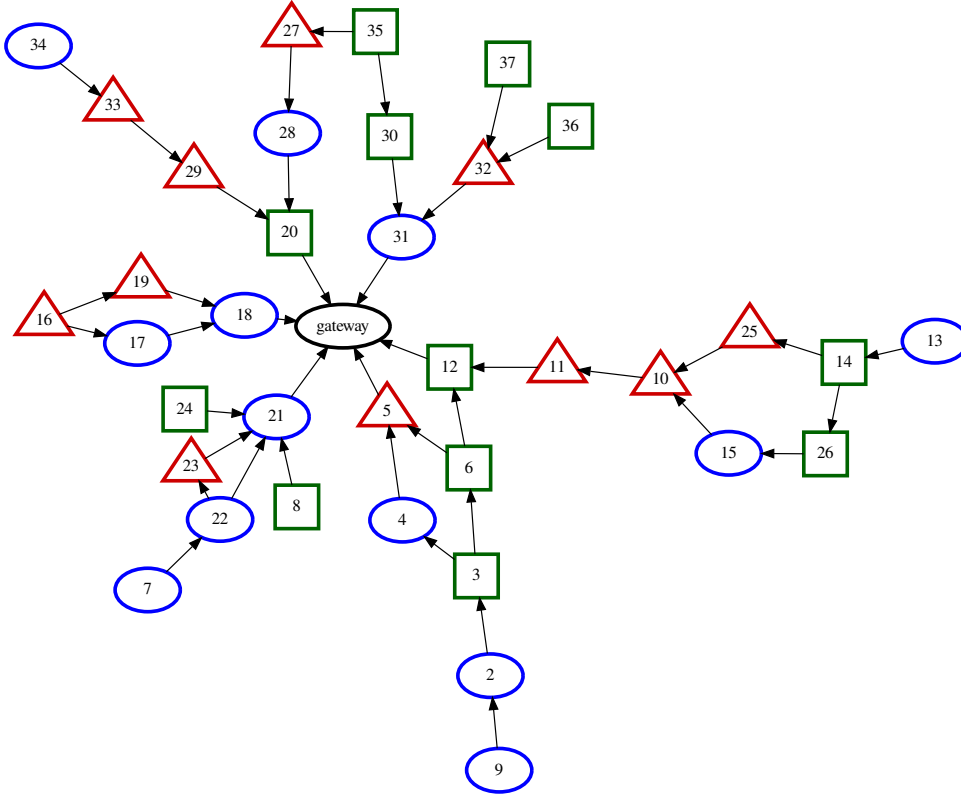


Figure 2.4: Network layout of the DeltaIoT.v2 network

Figure 2.5 shows the adaptation space of the different possible configurations. Each dot in this graph represents a separate adaptation option, i.e., in this case a network configuration.

2.2.3 Common characteristics

In both networks we have to deal with similar uncertainties. On the one hand we deal with the noise that is present during the transmission of packets over the network. This noise can heavily fluctuate on certain links, as can be seen in figure 2.6a. On the other hand we have the amount of messages that are generated by the individual motes. A temperature sensor measures the temperature periodically and as a result generates data quite consistently. An RFID sensor however does not share the same

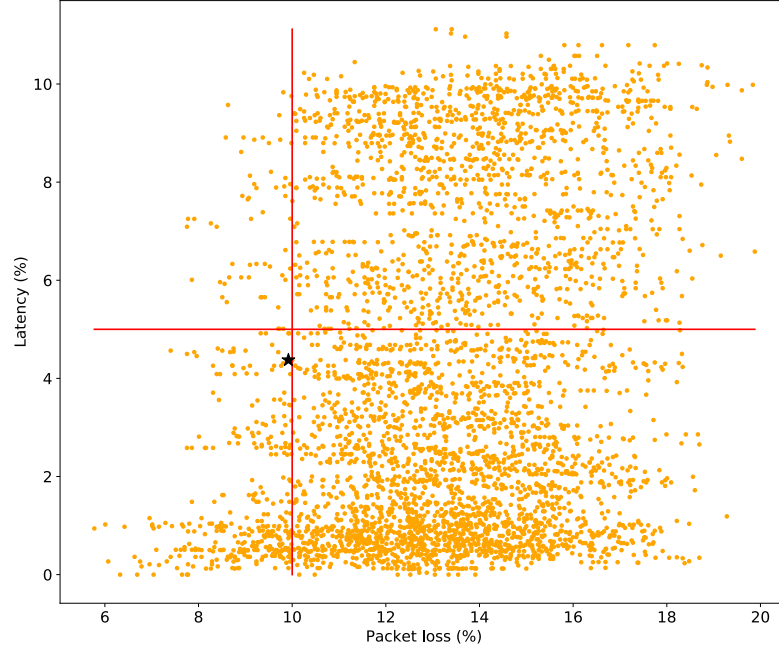


Figure 2.5: Adaptation space in DeltaIoT.v2 for a single network cycle

behavior. The amount of traffic generated from those sensors are dependent on the time of day. During the day messages are more frequently generated than during the night. These two uncertainties fall under the type *Environmental uncertainties*. These uncertainties fall under the *Aleatory* nature as mentioned in section 2.1.2.

An example of how these uncertainties can influence the qualities of a single adaptation option is illustrated in figure 2.6b. The configuration in this figure is the one highlighted with a black star in figure 2.5. We tracked this configuration over 10 cycles. As we can see, the qualities of a single adaptation option do not remain static due to the uncertainties encountered at runtime.

The noise in the environment and the traffic generation both have an effect on the qualities of the system. More noise means that either (A) more packets get lost during transmission or (b) the power setting of a link has to be increased resulting in higher energy consumption in the system. A spike in traffic generation can cause higher latency in the system (packets are held up since motes can only send a limited amount of packets in their time slots), as well as resulting in loss of packets due to buffers overflowing in motes. It is therefore really important to keep on top of these uncertainties and deal with them appropriately.

To deal with these uncertainties and consequently maintain the goals of the system, we can adjust the system configuration. This includes tuning the power settings of the different links (values in interval $[1, 15]$) and adjusting the message distribution

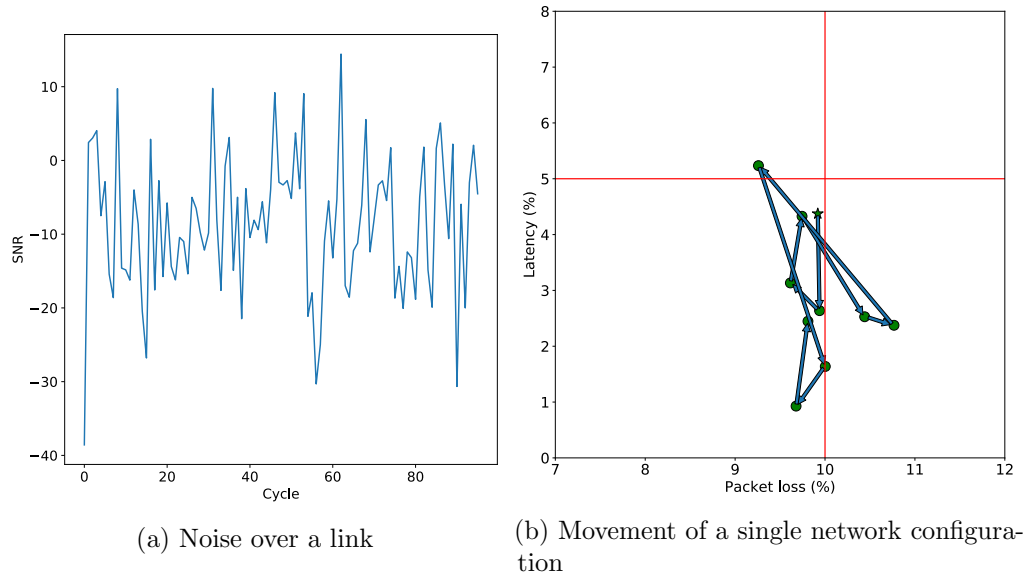


Figure 2.6: Influence of uncertainties on qualities of a single configuration

(denoted in percentages) in case a mote can send packets to more than one parent. The power setting mitigates the potential loss of packets during transmission at the cost of consuming more energy from the mote's battery. Message distribution can mitigate both packet loss and latency concerns: packet loss by sending packets on a route with less noise and latency by sending packets to a lesser occupied route (in terms of packet throughput). For message distribution however it is not necessarily a trade-off between energy consumption and other qualities. Even though sending packets on a longer route might not seem energy efficient at first, it could very well be the case that the power of transmission required on the busier route is significantly lower than on the shorter route. In other words, the message distribution settings in the network are quite versatile in terms of their effect on the different quality attributes of the system.

Figure 2.7 shows how the configuration settings of two motes could look like. The configuration of a single link is expressed in the form of a tuple. The first value in the tuple corresponds to the power setting on that link. The mote has in this case configured a power setting of 5 for the upper link and 9 for the link below. The second value in the tuple corresponds to the message distribution. The upper link is configured to handle 66% of the traffic while the link below handles the remaining 34%.

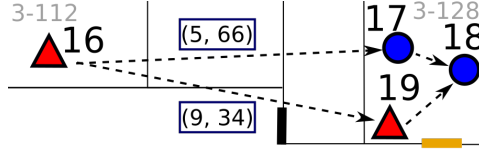


Figure 2.7: Example configuration of two links in the DeltaIoT.v2 network

We use three user goals that have to be realized in the IoT networks, regardless of the uncertainties present in the system. To that end, adaptation is necessary to switch to a different configuration if one of the goals is violated. Below we list the different quality requirements of the networks:

R₁: The average packet loss over 12 hours should not exceed 10% of the messages sent;

R₂: The average latency over 12 hours should not exceed 5% of the cycle time;

R₃: The average energy consumption over 12 hours should be minimized.

2.3 Quality models

In this research we make use of Uppaal [25], a tool produced at Uppsala University Sweden and Aalborg University Denmark. This tool provides an environment to model and verify formal models. The models are expressed in timed automata. Timed automata are an extension of finite automata by also integrating an internal clock in the automata. The models that were developed do not make use of this internal clock, but rather use a discrete clock to keep track of the cycles and time slots in the system. It is possible to verify models in two ways: simulation and statistical model checking. When using simulation, the model is simulated for a given amount of time specified by the developer. The results of this simulation can then be read and used. Alternatively, statistical model checking reasons about the probability of a model reaching a given state in the model. This probability can then likewise be used to reason about the quality of the system.

In the following sections we will discuss the quality models used to predict the qualities of the networks. The structure of the models remain the same between DeltaIoT.v1 and DeltaIoT.v2. A few parameters (such as for example buffer size) are adjusted between the models of the two networks.

2.3.1 Packet loss

The first model we use is a model to estimate the packet loss of a configuration, which is shown in figure 2.8. The model starts in the `InitialMoteChoice` automaton. In this automaton we choose, as the name suggests, a starting mote. After

choosing a mote the probability of that mote sending traffic gets evaluated. If traffic gets generated we move on to the second automaton **Topology**. Here we simulate the route from the initial chosen mote up until the gateway. If the mote has 2 links, a choice is made dependent on the message distribution settings of both links. Once a link is chosen we move on to the third automaton **Network**. In this automaton we calculate the packet loss probability of the link by analyzing the power setting of that link. If the packet is lost during transmission the model is stuck in the **PacketLoss** state. Otherwise the model resumes evaluation from the receiving mote, up until the packet reaches the gateway if it is not lost on the way.

The evaluation of the model happens through statistical model checking. The following query specifies how we extract the packet loss in the network:

Pr [≤ 1] ($\langle \rangle$ **Network.PacketLoss**)

The query evaluates the probability of reaching the **PacketLoss** state of the last automaton. The probability of reaching this state is similar to estimating the packet loss of the configuration in the network. This evaluation can then be repeated for multiple network configurations.

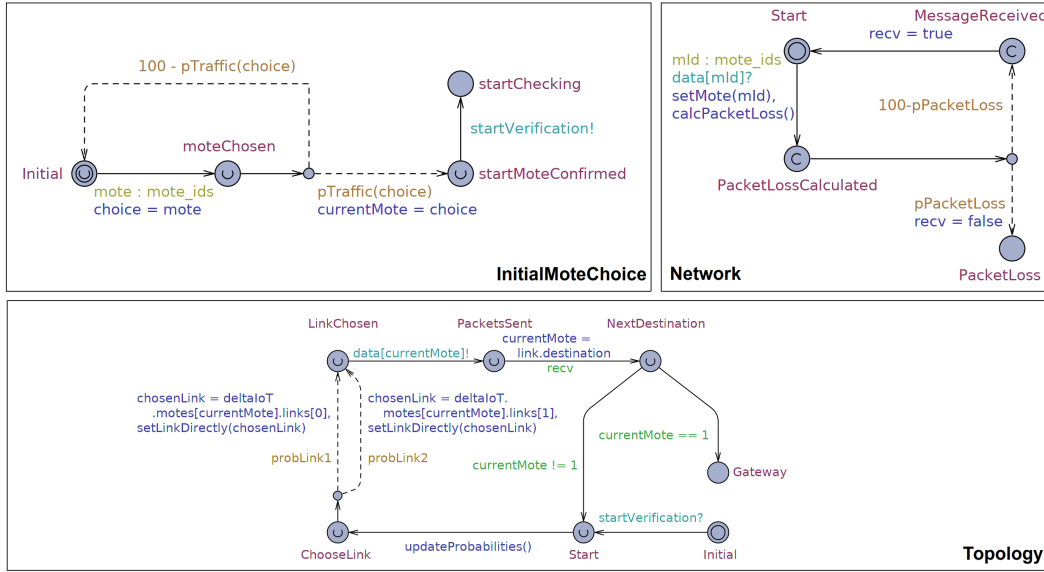


Figure 2.8: Formal model used to calculate the packet loss of the network

2.3.2 Latency

The second model we use estimates the latency of the network. The automata of the model can be found in figure 2.9. The model starts in the **StartVerification** state in **System**. From here, the simulation of a complete cycle in the simulator is triggered. Each mote is individually simulated in the **MoteAutomaton** automaton. The model determines if the mote generates packets according to its probability of generating traffic. Previously buffered packets, received packets and newly generated packets

2. SELF-ADAPTATION

are then sent over the outgoing links of the motes. The receiving motes place these packets inside their internal buffers for later transmission. At the end of the cycle the gateway is simulated in the **GateWay** automaton. At this stage the total latency of the system gets calculated by analyzing the total amount of packets sent and received.

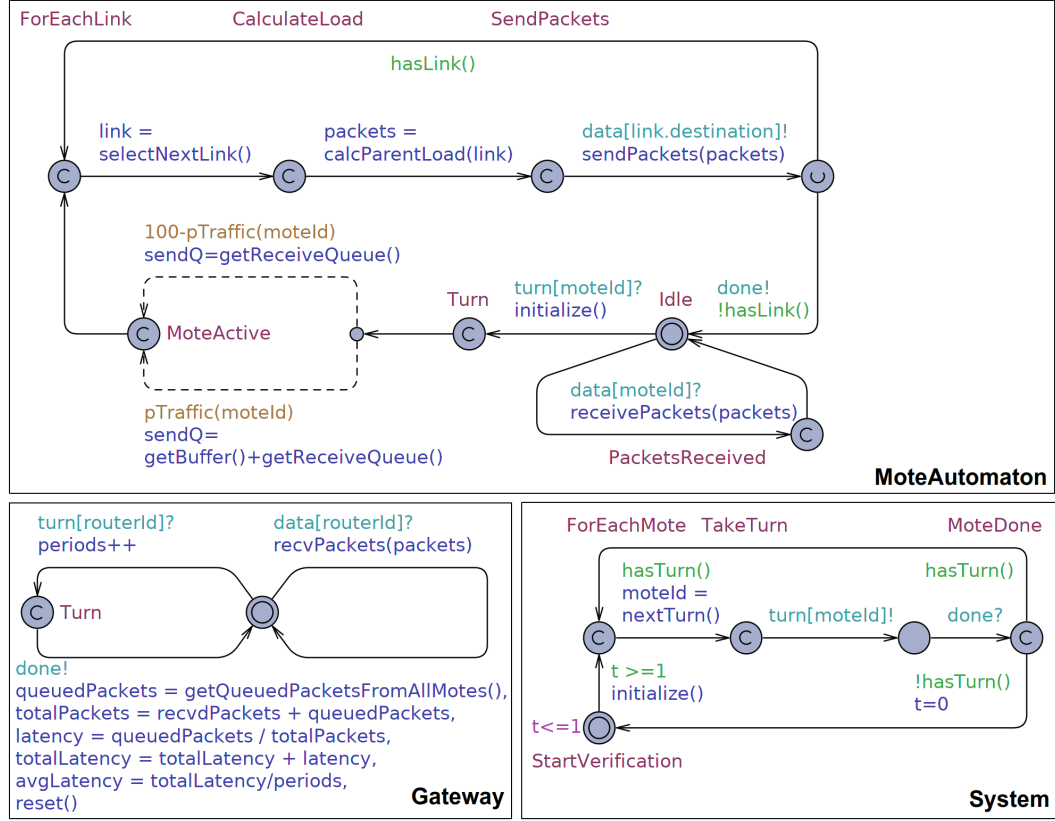


Figure 2.9: Formal model used to calculate the latency of the network

Verification of this model is done via simulation. We use the following query:

```
simulate 1[<=30]{Gateway.avgLatency}
```

Multiple cycles are simulated in the network. At the end of each cycle the latency of the network is calculated. The average latency is then calculated from all previously simulated cycles. This average latency is then used as a prediction for the latency of the network for a specific adaptation option.

2.3.3 Energy consumption

The last model we use is to estimate the energy consumption of the network for a specific adaptation option. This model, as shown in figure 2.10, is constructed very similarly to the latency model described previously. Simulation cycles are fully simulated with the addition of keeping track of how much energy is used for sending and receiving packets. This energy is added together at the end of a simulation cycle. We use the following query to evaluate the model:

```
simulate 1[<=3]{Gateway.avgEnergyConsumption}
```

Multiple cycles are simulated and, at the end, the average energy consumption per cycle is evaluated. This process is repeated for the different adaptation options in the same way as the other quality models.

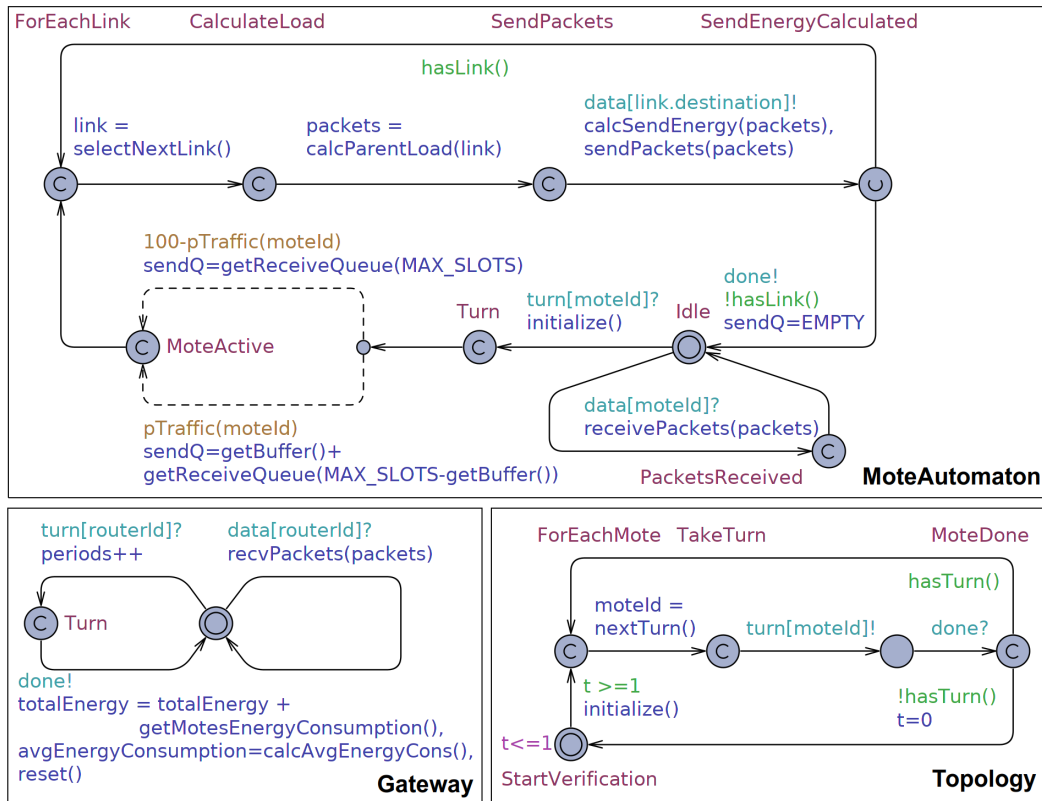


Figure 2.10: Formal model used to calculate the energy consumption of the network

2.4 Conclusion

In this chapter we have discussed the basic principles and properties of self-adaptation, and why it is necessary in systems which deal with uncertainties at runtime. After this we explored two internet of things networks which we use as a case study in our evaluation later on. These networks require self-adaptation in order to retain their quality goals at runtime. To conclude, we discussed the formal models used to analyze non-functional qualities of the two IoT networks. These models allow the self-adaptive system to reason about the desired qualities at runtime.

In the following chapter we go over the problem statement of large adaptation spaces in more detail, and propose a methodology to deal with this problem accordingly.

Chapter 3

Methodology

In this chapter we describe our approach to tackle the problem at hand. Firstly we delve deeper into the problem statement that was briefly discussed in chapter 2. Concretely, we will describe the situation where a standard MAPE-K loop is used to adapt the DeltaIoT.v2 network, and why this way of working is not feasible. In the following section we go into the proposed methodology to solve this issue with a machine learning module. We go over how the proposed solution fits in the current MAPE-K architecture and how this relates to the standard MAPE-K loop. Lastly, we illustrate how the proposed methodology can be used in our case study networks by using two well known machine learning techniques: classification and regression.

In chapter 4 we discuss how the machine learning module is set up and how it should behave during runtime. Chapter 5 goes over the evaluation of classification and regression with a concrete setup and simulation of the machine learning module.

3.1 Problem in detail

In the previous chapter we have briefly brought up the problem statement that we are tackling in this research. The introduction of formal techniques to provide guarantees about the (non-functional) qualities of the system does not go hand in hand with scalability of these approaches. Systems which operate on a smaller scale, i.e., systems which have to deal with a limited amount of adaptation options, still have the option of exhaustive formal verification of all the different adaptation options in the system.

A system on a larger scale however, where the time necessary to formally verify all the adaptation options grows quite large, does not have this ability anymore [27]. More concretely: the time needed to formally verify the adaptation options and find an optimal adaptation option exceeds the available time to adapt the system. In this case, we have to reduce the time it takes find a new system configuration to be bound by the available time.

3.1.1 DeltaIoT.v1

The first network that we will study has the possibility of formally verifying all the different adaptation options, since the adaptation space and the size of the network are still manageable. In this network we have the option to adjust the energy settings and the distribution of messages over each link to achieve the desired quality goals in the system. The adaptation space is thus built by varying these parameters. We will adjust the power settings of the links each cycle so that the noise value of those links is minimal.

We let the distribution of messages for motes with more than one parent vary with a factor of 20. Concretely, we construct the following possible distribution settings for the motes with two parents: 0/100, 20/80, 40/60, 60/40, 80/20, 100/0. DeltaIoT.v1 (as described in chapter 2) has three motes which have two parents each. Following this, we get an adaptation space with $6^3 = 216$ options. Formal verification of these options takes a little over 30 seconds on average ($\approx 0.14s$ / option). We have 480 seconds available each cycle to make a decision about the new configuration for the network, which means that it is possible (and optimal) to apply an exhaustive approach for this network.

3.1.2 DeltaIoT.v2

The second network that we study is a larger network that houses 37 motes. The composition of the adaptation space for this network is very similar to that of DeltaIoT.v1. The main difference between the networks lies in the size of the adaptation space and the size of the networks themselves. The effect of this is that the time required to verify the qualities of the different adaptation options differs greatly.

On the one hand, the network has six motes which have two parents each. The distribution of the messages varies with a factor of 34. This gives us the following possible combinations in terms of distribution settings: 0/100, 34/66, 68/32, 100/0. The energy settings are adjusted each cycle in a similar manner as described in the DeltaIoT.v1 case. This gives us a total amount of $4^6 = 4096$ different adaptation options in the adaptation space.

On the other hand, the size of the network also has an effect on the time required to verify the qualities of the network using formal models. The network consists of quite a few more motes and links compared to DeltaIoT.v1 which have to all be taken into account when performing formal verification using the models. Total verification of all the different adaptation options takes 750 seconds on average ($\approx 0.18s$ / options). The system however only has 570 seconds to choose a new configuration. A decision about this new configuration thus needs to be made in these 570 seconds. With this we can conclude that exhaustive formal verification in this network is not feasible anymore.

3.2 Architecture

In order to solve the problem of not having enough time to verify all the adaptation options, we propose to extend the analyzer of the MAPE-K feedback loop with a new module: a machine learning module. The purpose of this new module is to reduce the adaptation space to a smaller more manageable subset of adaptation options that are deemed to be relevant. Relevance in this context means that the options would have been considered/eligible in the selection process to become the new system configuration. For example: the module might only consider the options for selection which comply with at least a fixed amount of goals in the system. The machine learning module is, in this research, limited to dealing with threshold goals.

Figure 3.1 describes the architecture and the workflow of the MAPE-K loop without the proposed machine learning module. All the different adaptation options are formally verified by the analyzer and sent forward to the next phase of the MAPE-K loop (planner).

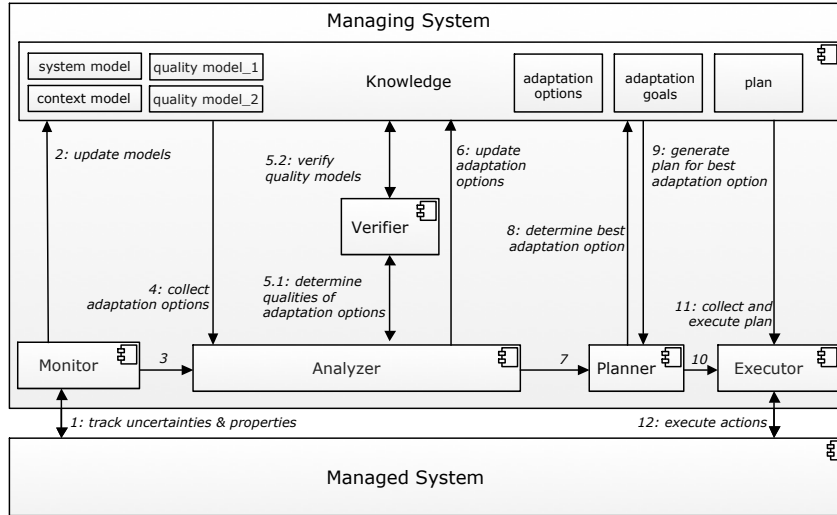


Figure 3.1: Original architecture and workflow of the analyzer

Figure 3.2 shows how the machine learning module is added to the standard architecture and process of the analyzer. The other phases of the MAPE-K loop remain unchanged. The *Analyzer* start with collecting the different adaptation options from the *Knowledge* (4). These options are then passed on to the *Machine learning module*. The machine learning module predicts which of the collected options are relevant (5.1). This prediction is done by making use of the trained model of the machine learner (5.2). The qualities of the relevant subset of adaptation options get formally verified by a *Verifier* (6.1). This verifier makes use of quality models which include the inner workings of the system that are relevant to the specific qualities (6.2). The subset of verified options are then updated in the *Knowledge* (7). These options are

3. METHODOLOGY

also sent back to the machine learning module for online learning (8).

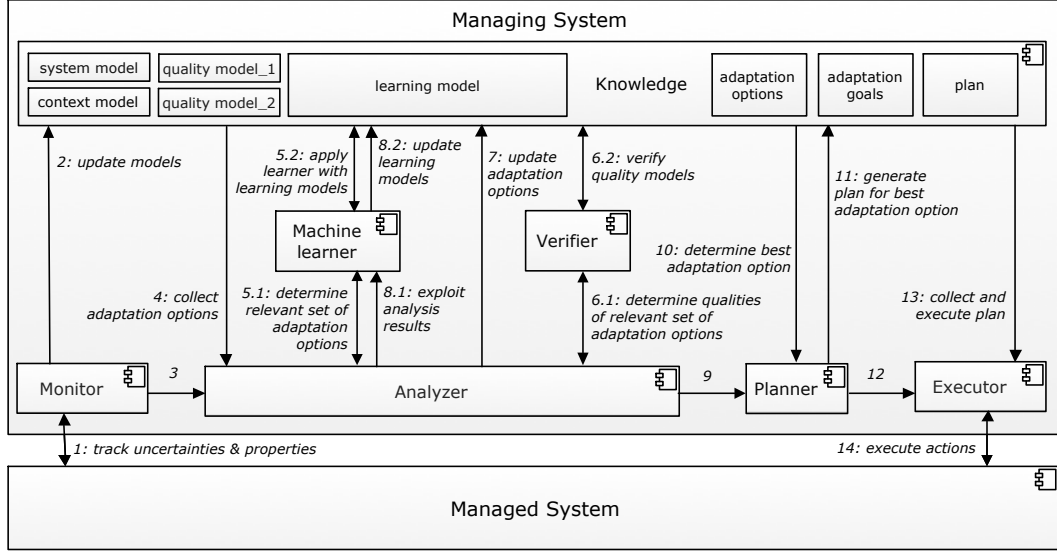


Figure 3.2: Architecture and workflow of the analyzer extended with a machine learning module

An important aspect of this solution is the need of a machine learning technique that support online learning. Online learning, also referred to as incremental learning, is a technique of learning where the machine learner can learn multiple times without discarding learned information from old training data. This property is crucial to self-adaptation for several reasons: e.g. self-adaptive systems need to be able to handle uncertainties that can only be solved at runtime. This means that the values of those uncertainties are not predictable beforehand, making it impossible to train the machine learner fully beforehand. If new uncertainty values are encountered at runtime, we want our machine learner to capture this new information in its future predictions.

3.3 Concrete application to DeltaIoT

In this section we apply the proposed methodology, as described in the previous section, in practice to the two DeltaIoT networks. We illustrate how the machine learning module can be instantiated by two different machine learning techniques: classification and regression. The implementation of the module is discussed in chapter 4. The analysis of the retrieved results of these instantiations and the implementation is described in chapter 5.

3.3.1 Machine learning techniques

Classification

Classification [28] is a supervised machine learning technique that attempts to classify an observation to a class (out of a predetermined set of classes). In our case study networks, we define an observation as follows: a combination of uncertainties which are measured in the network at runtime and an adaptation option of the system. The uncertainties in the system are on the one hand the noise over the individual links in the network and on the other hand the probabilities that motes send messages. The adaptation option is composed of the energy settings of the individual links as well as the message distribution settings of the different links.

We construct the classes for classification by compliance with the user goals of the system. Here we make a distinction between the two different types of goals in a system. Threshold goals can be mapped to classes in the following way: define 2^n classes (n being the amount of threshold goals) which all represent different combinations of either complying or violating a user goal.

Optimization goals on the other hand are more difficult to define. The output of classification is a discrete variable, i.e., class label, while in this case we need a continuous variable in order to evaluate the optimization goal. A potential workaround could be to discretize the values of a desired quality into multiple intervals. These intervals could then be used as the different classes for a single optimization goal. This approach however could result in a lot of different classes when dealing with multiple optimization goals. The approach also assumes that there is sufficient knowledge of the system beforehand in order to learn the spread of values for these qualities. Discretizing these quality values is not straightforward if this knowledge is not fully acquired or known at design time.

In the two networks that we study we make predictions about the packet loss and latency values of the adaptation options. These two qualities relate back to two threshold goals in the system (as described in 2.2). We define the classes for these two goals as follows:

C₀: No goals are satisfied.

C₁: Only the packet loss goal R_1 is satisfied.

C₂: Only the latency goal R_2 is satisfied.

C₃: Both goals R_1 and R_2 are satisfied.

Figure 3.3 shows how the machine learning module can be specifically instantiated for classification. This module is filled in with a concrete combination of a classifier and a scaler. The choice of such a combination is further discussed in chapter 4. The classifier and scaler both have its own model where they keep track of already

3. METHODOLOGY

learned knowledge. The model of the scaler stores, dependent of the scaler type, specific properties of the learned observations (such as e.g. maximum values). The model of the classifier holds information about the learned target function which maps observations to one of our predetermined classes.

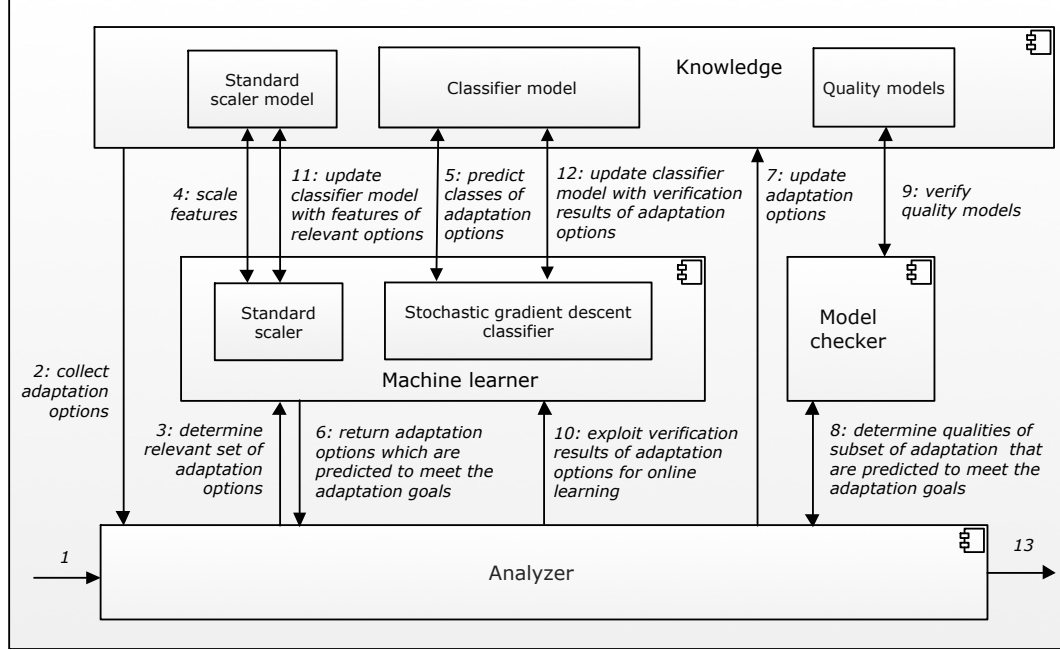


Figure 3.3: Architecture and workflow of the analyzer, filled in with a scaler and classifier

Regression

Regression is a supervised learning technique that attempts to predict the value of a dependent variable by analyzing an observation. Similarly to classification, regression attempts to learn a target function with maps the input data, i.e., observations, to an output variable (the dependent variable). This output variable is a continuous variable as opposed to classification which uses a discrete class label. The observations in our case study remain the same as the observations defined in the previous section for classification. The dependent variables in this case are the values of the desired qualities of the system for the different adaptation options.

We can use the predictions made by the regressor(s) to filter out adaptation options which do not comply with certain user goals. For threshold goals we can directly use the predicted values of the qualities to check if the user goal is fulfilled. Optimization goals are also handled more easily compared to classification. Since we are dealing with continuous variables in this case, we can just use these values in the goals. Regression is, for that reason, a more suitable solution when the system requires

that the machine learning module also has to deal with optimization goals.

Figure 3.4 shows the concrete instantiation of the machine learning module for regression. The diagram and the workflow in this figure are a little more succinct since they follow the same workflow as in figure 3.3. The choice of regressor and scaler are discussed in chapter 4 as well.

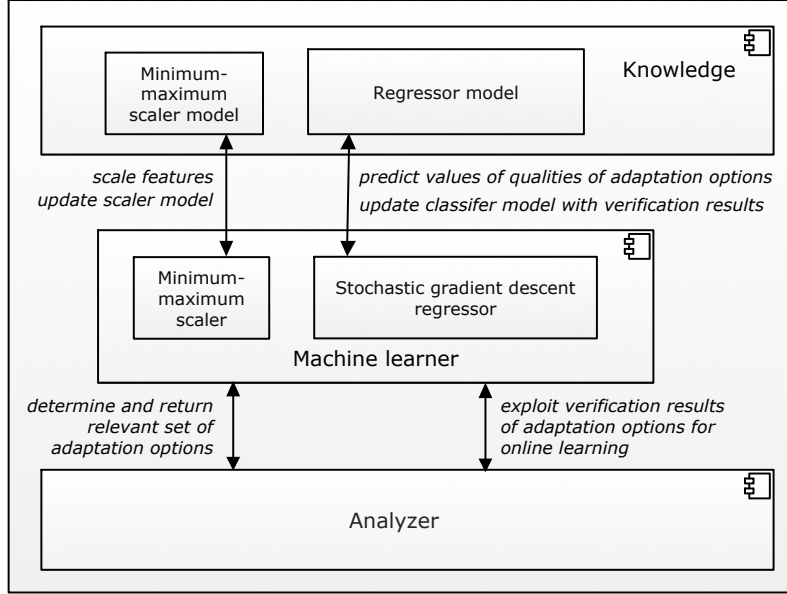


Figure 3.4: Architecture and workflow of the analyzer, filled in with a scaler and regressor

3.4 Conclusion

Formal verification in self-adaptive systems can, when the system and/or adaptation space is large, be very time and resource consuming. This in turn can result in the infeasibility of the system to perform complete analysis of all the different adaptation options.

In this chapter we proposed an extension to the *Analyzer* component of the classic MAPE-K feedback loop: a *Machine learning module*. This machine learning module supports the analyzer by filtering out adaptation options which do not comply with user goals in the system. Lastly, we described two approaches that can be used to fill in the machine learning module: classification and regression. For classification we constructed four different classes which cover the different possibilities in compliance with the two threshold goals. Regression can be directly used to predict the values of the desired qualities in the system, and thus directly checking which options do comply with the goals of the system.

Chapter 4

Implementation

In this chapter, we discuss how the proposed methodology from the previous chapter is specifically implemented for the two case study networks. Along with this, details of how the experiments are conducted are further discussed. In the following chapter we evaluate the results we have gathered from these two networks.

4.1 DeltaIoT simulator

The networks, as discussed in chapter 2, are both deployed in physical environments. Since data collection and evaluation of experiments take up a lot of time in these systems, a simulator has been developed. We will call this simulator the *DeltaIoT simulator*. The simulator is written in Java and is modeled as closely as possible to the physical networks.

The simulator adheres to the topology of both networks. Each mote generates a certain amount of packets (dependent on the setting for that mote) each cycle. The mote transmits these packets, as well as buffered packets from a previous cycle, further down the network. The specific link(s) on which the packets are transmitted depends on the configuration of the network. The order in which motes transmit their packets is determined by the network topology.

During transmission of packets, the simulator keeps track of information related to the qualities of the network. An example of information that the simulator tracks is the amount of packets which are lost due to a bad Signal to Noise Ratio (SNR). This information is then analyzed at the end of a simulation cycle. As a result of this, the qualities of the network are easily measured and evaluated within the software. Four different qualities are currently measured: packet loss, energy consumption, latency and queue loss. In this research, we will focus on the first three.

A MAPE loop is deployed on top of the simulator. In this case, the simulator is the managed system which we observe and adjust/reconfigure if deemed necessary by the managing system. The monitor component tracks the uncertainties in the

networks: the SNR values of the links and the load of the nodes. This information is then used by the analyzer. The analyzer uses formal models (discussed in more detail in chapter 2) and the gathered knowledge to determine the qualities of the adaptation options. The planner chooses an optimal configuration for the network and constructs a plan for the system to adjust to this new configuration. Finally, the executor makes sure this plan is executed in the simulator.

As proposed in chapter 3, we also enhance the analyzer component of the MAPE loop. The specific configuration and setup of this component is discussed in section 4.2. Section 4.3 describes how the machine learning component is used at runtime.

4.2 Offline activities

We define offline activities in the following way:

"A set of activities that has to be completed before a system can be deployed and used at runtime."

Concretely, the offline activities are the steps that have to be completed in order to set the system up. Once these activities/steps are finished, the system can be configured accordingly. If we apply this definition to both DeltaIoT networks, without considering the machine learning component for now, the offline steps are the following:

- Development of the MAPE loop which runs on top of the managed system.
- Determining the parameters for this MAPE loop (e.g. under which conditions do we consider adapting the system, how is our adaptation space composed, ..).
- Specification and implementation of formal models which can predict the qualities of a specific adaptation option in an uncertain environment.

When these steps are concluded, the managing system can be deployed and used at runtime. The MAPE loop and its configuration make sure the managed system is adapted according to the desired user goals. The formal quality models are used at run time to aid the managing system in choosing a new adaptation option, as well as reducing the complexity of the system by separating this logic from the MAPE loop itself.

In the following subsections, we discuss the different offline steps which have to be completed in order to prepare the machine learning module. The outcome of this process will be a specific machine learning model as well as the values for a few parameters which will be used at run time. Chapter 5 gives an outline of this process for the DeltaIoT networks.

4.2.1 Data collection

The first offline activity is data collection. This data is required for a number of analytical tasks that need to be performed to set up the learning module. In our context, we need data of the different qualities for all the adaptation options over a number of cycles. The data is composed of the *verified qualities* and of the *features* of the adaptation options. Features are variables in the system that affect the predicted qualities of adaptation options. This includes the uncertainties in the network (network interference and load) and effector settings (power settings over the communication links and distribution of messages sent to parents).

A simplified example of the format of the data is described in table 4.1. Each line in this table represents a single adaptation option, i.e., the combination of all the features, and their respective qualities. This raw data is then used in the next step of the offline activities: feature selection.

Noise link 1	Noise link 2	Power link 1	Power link 2	Packet loss
25	-3	1	1	16%
8	-5	5	5	8%
12	-15	5	15	14%
-3	-5	12	5	3%
0	-7	3	8	5%
...	...	...	...	...

Table 4.1: Example format of some collected offline data

4.2.2 Feature selection

The next step that is required is called feature selection. In this step we use the previously collected offline data from section 4.2.1. Feature selection evaluates the effects that different features have on the actual qualities of the adaptation options. The effect of the features is expressed by giving each feature an importance score. A feature selection algorithm calculates this score automatically, allowing this step to be completely automated without human intervention.

Features with a very low score are considered not relevant to the predictions of the machine learner. In this work, we have filtered out these features, so they are not used at runtime by the machine learners. Feature filtering helps simplifying the model of the machine learner and empirical evaluation showed that it also improved the accuracy of the learner. It is important to note however that a potential disadvantage of this approach is that we may have missed relevant features due to the specific coverage of the training set.

Table 4.2 gives a simplified example of what the resulting data could look like after performing feature selection on the data from table 4.1. The feature selection algorithm determines that *Feature 3* does not have a significant impact on the qualities that we want to evaluate. For that reason, the feature can be pruned from the data set.

Noise link 1	Noise link 2	Power link 1	Power link 2	Packet loss
25	-3	1	1	16%
8	-5	5	5	8%
12	-15	5	15	14%
-3	-5	12	5	3%
0	-7	3	8	5%
...	...	...	...	...

Table 4.2: Example of performing feature selection on the data from table 4.1

4.2.3 Model selection

The third offline activity is model selection. In this activity, multiple combinations of machine learners (with variations in their internal parameters, optimization functions, ...) and (in our case) scalers (which transform features to a specific scale) are evaluated. It is important to note that this evaluation is done with only the relevant features determined by feature selection as mentioned before. The features that are deemed irrelevant are not used at this stage.

During model selection the designer evaluates different combinations of machine learners and scalers by utilizing a number of quality metrics that are further discussed in chapter 5. A simple example of a metric that could be used to evaluate these different combinations is the *accuracy* of the predictions. Using a portion of the data gathered from step 1, we train the model at first. The size of this portion is one of the parameters judged in this step, meaning different training sizes are evaluated. After training the model, the relevant features of the remaining data, i.e., features of the data that is not used for learning, are used to make predictions about these configurations. The accuracy of the machine learner here is the amount of right predictions relative to the total amount of predictions made.

The result of this activity is a selected machine learner, a scaler and different configuration parameters that are used to support efficient runtime analysis. An example of a chosen model is shown in table 4.3. Three models are evaluated with their respective predictions over the different adaptation options. Here we choose the third model for example. Once this step is completed the system can be configured accordingly to be used at runtime. Figure 4.1 gives an overview of the different steps we just described.

Packet loss	Packet loss < 10%?	Model 1	Model 2	Model 3
16%	0	0	0	1
8%	1	1	1	1
14%	0	0	1	0
3%	1	1	1	1
5%	1	0	0	1
...	...	...	...	...

	Model 1	Model 2	Model 3
Accuracy	80%	60%	80%

Table 4.3: Example of performing model selection

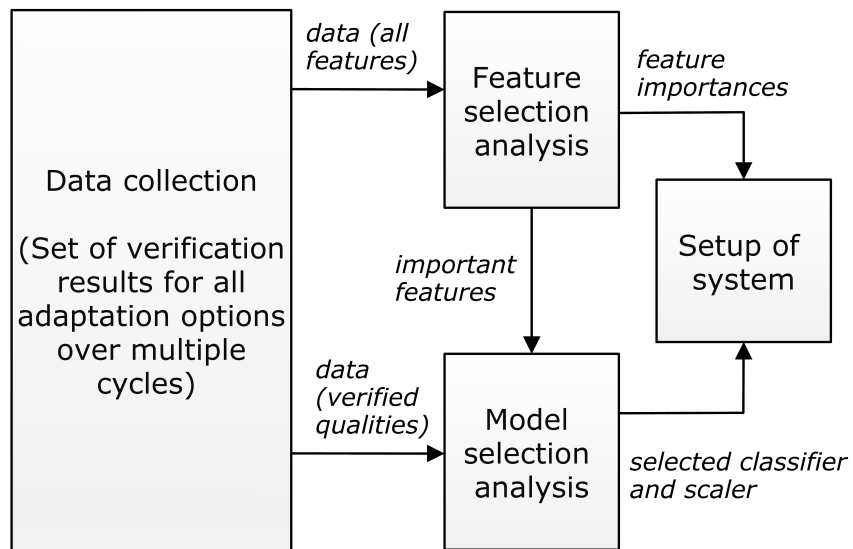


Figure 4.1: Diagram of the different offline steps, concretely for classification

4.3 Online activities

We define online activities as follows:

"A set of activities that take place when the system is deployed and operational."

Once the offline activities are concluded and the machine learning module is configured accordingly, we can start using the module to perform self-adaptation on the managed system. In each cycle of the network, the different steps of the MAPE feedback loop system are iterated and evaluated. We make a distinction between two types of cycles encountered at runtime: training cycles and testing cycles. In the following sections, we discuss these two types of cycles separately, as well as their roles in relation to the machine learning module. An important detail here is that, after the system has been set up, no human intervention is required. The steps described in the following sections are fully automated while the system is running.

4.3.1 Training cycles

The first phase we define is called a *training phase*. During these cycles, runtime data of the system and the environment is collected at first. This data is then used to update the runtime models. More specifically, we use the data gathered here to train the machine learning models. Naturally, these type of cycles only occur in the beginning stages of the system deployment. Later on, when the machine learner has learned a sufficient amount of data samples, we can continue on with the next type of cycles: *testing cycles*. The specific amount of training cycles that have to be run through is decided in the model selection process of the offline activities. Alternatively it is possible to use an ad-hoc way of evaluating the learner after each training cycle. This is however not discussed in this work.

Since the machine learner has not yet learned enough to make intelligent predictions in this case, the only option left is to provide as much learning data as possible to the learner. Concretely, we use the quality models (as discussed in chapter 2) to evaluate as many adaptation options as we can in the time we have each cycle. Because verification of all the options is not possible (given the problem definition we are discussing), only a subset of these options can be verified. To ensure fairness of verification across the adaptation options, we keep track of the last adaptation option that is verified in the training cycles. The following training cycle then continues on from this point, assuring that each option is verified and used for training equally without bias.

Now that we have the features and quality values of the analyzed adaptation options (similar format to the data in table 4.1), we can use this data to train the learner model. We use the information of the feature selection step in the offline activities (section 4.2.2) to only select the relevant features of the gathered data. The combinations of the relevant features and their corresponding qualities are then

leveraged by the learner to improve its model for feature predictions. During training, the adaptation space is not reduced since the learner is not sufficiently trained yet.

It is important to note that adaptation in training cycles is limited to the options which are verified by the formal verifier. Therefore the process of adaptation might not perform too well during these cycles (the selected options are not chosen intelligently, but rather put in a round robin style system). This is however somewhat unavoidable since the training of the machine learner attempts to solve this exact problem. Once the machine learning module is fully trained, we can start using it at its full capacity.

An alternative to the use of training cycles is to base the training on the analysis results of a selection of adaptation options per cycle (premature predictions by the machine learning module), or training the learner as a background process before testing starts. These options are not currently explored in this research.

4.3.2 Testing cycles

Once the learner model is sufficiently trained, the cycles from that point forward become testing cycles. In testing cycles, the knowledge learned by the machine learner is utilized by making predictions of the encountered adaptation options. Dependent on these predictions, a subset of adaptation options in the adaptation space can be selected for further formal verification.

At first, similarly to the training cycles, runtime data of the system and the environment is collected in the knowledge component of the MAPE loop. As opposed to verifying as many options as possible in the training cycles we can now use the machine learner to make predictions first. The features are thus sent to the machine learning module. Before they arrive at the machine learner itself, the values of the features are rescaled by the scaler component. The machine learner then makes a prediction about the qualities of the adaptation options after the transformation of the features.

When using classification, the classifier will map the features of an adaptation option to a class from a predetermined set. We define this predetermined set as all the different options in terms of complying with the threshold goals of the system. An adaptation option is therefore mapped onto the compliance of (multiple) adaptation goals. The specific classes we use are discussed in section 3.3. Regression on the other hand allows us to use the predicted values for the qualities to directly evaluate the goals of the system.

At this point we have a collection of features of the adaptation options with their respective predictions from the machine learning module. Algorithm 1 describes how the selection of adaptation options for formal verification is performed when using classification. `selected_options` is the set of adaptation options which is

deemed (most) relevant by the classifier. This set of options is always analyzed first. **explored_options** is a small subset of options which are predicted to meet less user goals. We consider this small subset to potentially correct the learning model in case it has predicted these options wrongly. In such a situation, the model gets corrected afterwards. The variable **exploration** denotes the percentage of options explored in this way. A similar approach is applied when using regression, by converting the predictions from the regressor to equivalent classification labels. The algorithm can then be reused for this approach as well.

Algorithm 1 Selection of adaptation options for verification

```

1: exploration = 0.1
2: best_options = []
3: explored_options = []
4: predictions = classifier.makePredictions(adaptation_options)
5:
6: if predictions.contains(C3) then
7:   best_options = adaptation_options.class(C3)
8:   explored_options = select_exploration(adaptation_options.class(C2) + adapta-
      tion_options.class(C1), exploration)
9: else if predictions.contains(C2) or predictions.contains(C1) then
10:  best_options = adaptation_options.class(C2) + adaptation_options.class(C1)
11:  explored_options = select_exploration(adaptation_options.class(C0), explo-
      ration)
12: else
13:   best_options = adaptation_options
14: end if
15:
16: analyzer.verifyAdaptationOptions(best_options + explored_options)

```

Once the subset of options are determined with the help of the machine learning module, formal verification, i.e., predicting the qualities of these options with the quality models, is applied. The **best_options** are verified first before the verification of the **explored_options**. This gives a priority to the most important options over the ones that are predicted to meet fewer goals. If the time to verify the subset still takes too long, it is important to give the best options priority over the lesser options.

The next and final step in the testing cycle is to use the verified qualities from the selected subset to adjust the machine learning module. In other words, the machine learner is able to adjust earlier predictions using the novel knowledge discovered and generated at runtime. Such adjustments may be required for potentially mis-predicted classes or values, and to exploit correlations between uncertainties and qualities that were not known before.

Table 4.4 gives an example (continuing the running example of section 4.2) of the

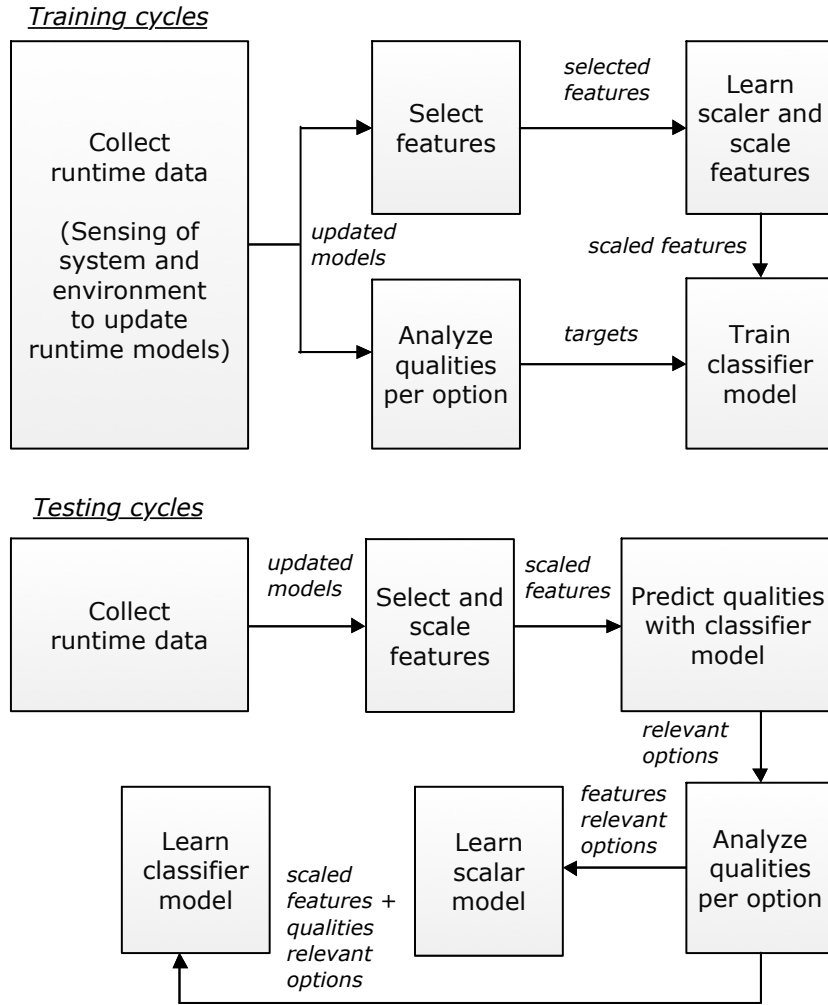


Figure 4.2: Diagram of the different steps in the testing and training cycles concretely for classification

different steps taken in the testing cycles that we have just discussed.

Figure 4.2 gives a summary of all the different steps that are taken during the training and testing cycles when using classification. The *updated models* label on the first edge in each cycle denotes updating the knowledge component of the MAPE loop with the relevant information of the underlying system (uncertainties and configuration).

4. IMPLEMENTATION

↓ Data collection

Noise link 1	Noise link 2	Power link 1	Power link 2
7	3	0	10
7	3	2	1
7	3	12	6
7	3	5	5
7	3	8	14

↓ Feature selection

Noise link 1	Noise link 2	Power link 1
7	3	0
7	3	2
7	3	12
7	3	5
7	3	8

↓ Model predictions

Noise link 1	Noise link 2	Power link 1	Prediction
7	3	0	0
7	3	2	1
7	3	12	0
7	3	5	1
7	3	8	1

↓ Formal verification & online learning

Noise link 1	Noise link 2	Power link 1	Prediction	Verification
7	3	0	0	-
7	3	2	1	1
7	3	12	0	-
7	3	5	1	1
7	3	8	1	0

Table 4.4: Example of the different phases in a testing cycle

4.4 Conclusion

In this chapter we discussed the different activities that take place before and during the system's operation. We defined the activities which have to be conducted beforehand as *offline activities*. Depending on the activity, human intervention and judgment is necessary. The result of these offline steps is a specific setup of the managing system. Once the managing system is configured accordingly, the *online activities* take place. Here we make a distinction between two different types of adaptation cycles: training and testing cycles. Training cycles are used to train the machine learning module with as much data as possible within the time constraint of the system. After sufficient training, the system goes through testing cycles. In these cycles, the machine learning module is fully exploited and used to select a subset of adaptation options via predictions. These options are then formally verified, considered for system adaptation and used to perform online learning.

In the next chapter we will discuss the results we have retrieved from applying this approach to the two DeltaIoT networks. At the end of the chapter, we will give an overall evaluation, as well as benefits and drawbacks of the approach.

Chapter 5

Evaluation

This chapter focuses on the evaluation of the proposed methodology and implementation suggested in chapter 3 and chapter 4 respectively. At first the results of the offline activities (discussed in section 4.2) are briefly covered for both networks and learning approaches. Next, we make a comparison between the machine learning approaches (classification and regression) and an exhaustive (unfeasible) approach by evaluating them in the two DeltaIoT networks. Here we also discuss the strengths and weaknesses of the solution. Finally we discuss threats to validity of this evaluation.

As a general note: we use the library *Scikit-Learn* [29] as part of our evaluation. This library provides implementations for different scalers, classifiers, regressors. Besides Scikit-Learn, we also make use of a few libraries to visualize data and plot graphs (e.g. *Matplotlib*).

5.1 Setup machine learning module

In this section we describe the way we conducted the offline activities as described in section 4.2. The first activity in this process is the collection of data before the system is run. For practical reasons, we collected this data from the simulator rather than the physical networks. We collected data over a period of one week (from 6 AM to 6 PM). This corresponds to data over 300 cycles with verified qualities and verification times of the adaptation options. The required amount of data was determined empirically based on the quality of the results obtained using learning.

The next step in the setup of the module was performing feature selection. For this part we used automated algorithms to calculate the importance scores of all the collected features in the data collection step. For classification we utilized the `ExtraTreesClassifier` classifier to calculate the previously mentioned scores. We use the classifier to fit the collected offline data, and extract the importance scores from this classifier. A similar approach is applied for regression, but with the use of the `ExtraTreesRegressor` regressor. Figure 5.1 shows the output we retrieved after evaluating the features of the collected data from the DeltaIoT.v2 network. In this

network, we have a total of 162 features. After feature selection, only 64 of the 162 features remain.

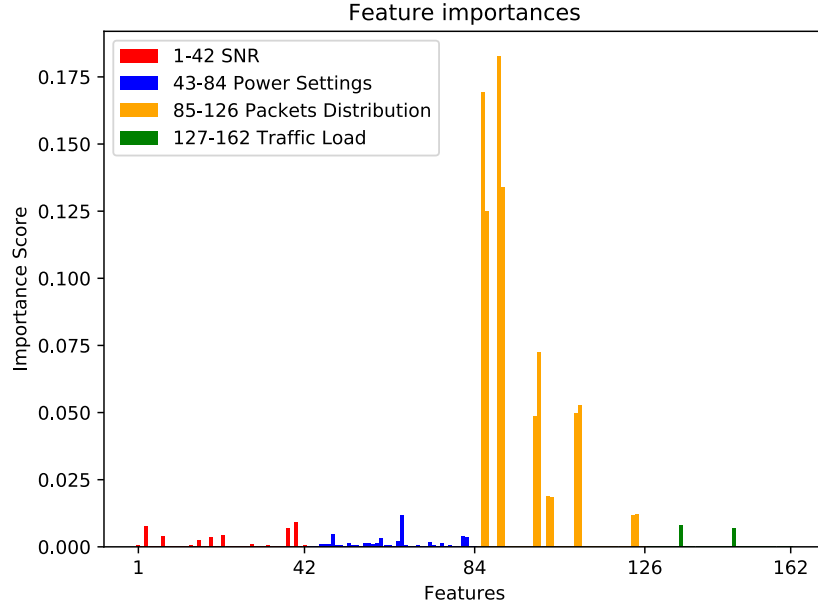


Figure 5.1: The importance scores of all the different features in the DeltaIoT.v2 network

The final step is model selection. In this step, we evaluate a set of different machine learning module configurations. Such setups include the learning model that will be used, the scaler which rescales the values of the features and two parameters. In the following subsections we will discuss the variations of these models/parameters.

5.1.1 Parameters

During model selection, we evaluated two parameters to be used at runtime. The first of these two parameters determines the **amount of learning cycles** that have to be completed before the machine learning module can be used to make predictions. After completion of these *learning cycles* (a.k.a. training cycles), every cycle in the system becomes a *testing cycle*. We evaluated this parameter for the following values: 30, 45 and 60.

The second parameter that was evaluated is the **exploration percentage**. As previously mentioned in section 4.3.2, we explore a few options which are predicted to comply with one less goal than the compliance of the best predicted options. The value for this parameter was evaluated for the following values: 5% and 10%.

5.1.2 Scalers

If we consider the different values features can take, it often happens that they are quite different. For example: in both DeltaIoT networks we use the power settings over the links as features. These feature values lay in the interval $[1, 15]$ (according to the possible power setting modes). We also use message distribution settings as features. These values are however expressed on a scale of 0 to 100 (expressed as percentages). Since these scales are quite different, the machine learner might have more difficulty learning the correlation between the features and the predictions, i.e., learning its target function. In order to mitigate this issue we introduce a scaler prior to the machine learner. This scaler rescales the values of the features so that the machine learner can learn more efficiently. We will discuss the different scalers which we considered below ¹. It is important to note that these scalers also require online learning capabilities. We do not know beforehand what kind of values some features (related to uncertainties at runtime) take on at runtime. Having the possibility to adjust the model of the scaler is therefore a necessity.

MinMaxScaler

The **MinMaxScaler** scales features, as the name suggests, according to their minimum and maximum values encountered over time. The scaler maps the values of the features to the interval $[0,1]$. The minimum and maximum values are updated at runtime if a lower/higher value is encountered.

StandardScaler

The **StandardScaler** scales features according to their respective means and standard deviations observed thus far. The scaler keeps track of the means and standard deviations for each feature and stores this information in the scaler model. At runtime, the scaler updates the means and standard deviations values taking into account new data via online learning.

MaxAbsScaler

The **MaxAbsScaler** scales features relative to the maximum absolute value encountered thus far. The feature values are mapped to the interval $[-1, 1]$. A feature that has the maximum absolute value will be mapped to a value of 1 (or -1 if the original value is negative). Features in between are scaled linearly to this maximum absolute value. Similarly to the other classifiers this maximum absolute value is updated at runtime when a new higher value is encountered.

¹We also considered the option of using no scaler. The results retrieved this way however were never better than one of the three scalers that are discussed.

5.1.3 Learners

The last variation in the model selection process is the machine learner itself. The machine learner is responsible for making the predictions about the adaptation options. It takes a set of features (representing the uncertainties and the adaptation options in the system) and predicts the compliance of one or more system goals judging by the values of these features. The scaler assists the machine learner in making this choice by adjusting the values of those features to a different scale. Below we discuss the machine learners we evaluated for classification and regression respectively.

SGDClassifier

The **SGDClassifier**, Stochastic Gradient Descent Classifier in full, is a linear classifier which makes use of stochastic gradient descent learning². This classifier estimates the gradient of the loss for each observation it receives by comparing the outcome predicted by the learner with the input sample, i.e., the learner predicts the class an adaptation option belongs to and compares this with the results obtained by formal verification. The loss represents the inaccuracy of predicting the category a particular observation belongs to. The model is then updated based on the estimated loss, improving the model and thus the target function.

PassiveAggressiveClassifier

The **PassiveAggressiveClassifier** [30] is a classifier that uses weight vectors to predict the labels of the input data. These weight vectors maintain weights associated with each feature of the adaptation options. Each time online learning is performed, the weights are updated if necessary. If the classifier convincingly classified a sample correctly, i.e., predicted the class of an adaptation option correctly with a high degree of certainty, the weights are not updated. This refers to the 'passive' part of the name. The aggressive part refers to the aggressive nature of weight adjustment if the samples are mis-predicted or predicted with inadequate certainty. This correction should then result in the correct prediction of the adaptation option with a high degree of certainty in the future.

Perceptron

The **Perceptron** classifier is a standard linear classifier. It shares a similar implementation as the **SGDClassifier**, but is a more specific instance of this classifier. For the interested reader we refer to [31–33].

GaussianNB

The last classifier which we considered is the **GaussianNB** classifier. This classifier, Gaussian Naive Bayes in full, makes use of Bayes' theorem as the name suggests.

²<https://scikit-learn.org/stable/modules/sgd.html>

The naive part in the name of the classifier reflects a simplification made to the calculation of certain probabilities in the algorithm. The classifier calculates the probabilities of an adaptation option belonging to the different classes and selects the class with the highest probability.

SGDRegressor

The **SGDRegressor**, Stochastic Gradient Descent Regressor in full, is a linear regressor that makes use of the stochastic gradient descent learning algorithm. We refer back to the **SGDClassifier** since the inner workings of this regressor is very similar.

PassiveAggressiveRegressor

The **PassiveAggressiveRegressor** is a regressor that uses weight vectors to predict a dependent output variable based on the input data. Similarly to the regressor mentioned before, we refer to The **PassiveAggressiveClassifier** since the implementation of this regressor is very similar as well.

5.1.4 Results Model selection

After composing all the different combinations of classifiers/regressors, scalars and parameter values (as mentioned in section 5.1.1), we had a total of 387 combinations for classification and 126 combinations for regression. These combinations include slight variations in the internal parameters of the classifiers/regressors (such as the used loss/penalty/... function). We evaluated all the combinations for the two IoT networks separately. In order to choose which combination was best suited, we calculated a few metrics.

The first metric we evaluated was the accuracy of the machine learners. Simply put, this metric evaluates the amount of samples the classifier/regressor predicted correctly relative to the total amount of predictions. The accuracy of the different learners varied from as low as 20% up until 85%.

A second metric we used is the Matthew's correlation coefficient. This metric produces a value in the range $[-1,1]$, where 1 refers to perfect predictions, 0 refers to predictions that are equal to random predictions, and -1 refers to completely incorrect predictions. A score higher than 0 in this case means that the learner makes more intelligent predictions than random predictions on the evaluated data. We measured scores in our evaluation between -0.01 and 0.7 .

Lastly, we evaluated the combinations based on their F1 score. This metric combines recall (the percentage of samples that were retrieved by prediction from a specific class) and precision (the fraction of samples that have been predicted to be of a specific class that are actually part of that class). The F1 score is defined as a value in $[0,1]$. A higher F1 score means in general a better performing classifier.

5. EVALUATION

For this F1 score, we looked at a combined score which covers all the individual classes, and at separate F1 scores per class ($C_0 \dots C_3$). The F1 score of C_3 (which we are most interested in) ranged from 0.2 to 0.9 for the different combinations of classifier/regressors, scalers and parameter values.

$$\text{F1-score} = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

After evaluation of the different metrics for all the combinations, we chose an individual setup for classification and regression in both networks. The models and parameter values are listed in table 5.1. The values of the metrics we measured for these combinations are listed in table 5.2.

	Learning cycles	Exploration percentage	Model	Scaler
v1 classification	45	5%	SGDClassifier	StandardScaler
v1 regression	45	5%	SGDRegressor	MinMaxScaler
v2 classification	60	5%	SGDClassifier	MinMaxScaler
v2 regression	60	5%	SGDRegressor	StandardScaler

Table 5.1: Chosen model combinations for classification and regression in both networks.

	Classifier v1	Regressor v1	Classifier v2	Regressor v2
Accuracy	79.93%	81.43%	86.81%	84.85%
Matthew's cor.	0.6565	0.6776	0.7003	0.6500
F1-score (comb.)	0.7922	0.7823	0.8624	0.8381
F1-score (C_3)	0.8585	0.8843	0.7301	0.7350

Table 5.2: Metrics for the classifiers and regressors of DeltaIoT.v1 and DeltaIoT.v2

5.2 Comparison exhaustive approach

To evaluate the online learning approaches for the DeltaIoT.v1 network, we simulate the running self-adaptive system for a period of five days (from 6AM to 6PM). We run the simulator on a computer system with an *i7 8550U 1.8 GHz* processor and *8GB RAM*.

For the DeltaIoT.v2 network, we have simulated the self-adaptive system for a period of four days (from 6AM to 6PM). This simulation was run on a stronger computer system with an *intel xeon skylake 8 core 2GHz* machine with *16GB RAM*.

5.2.1 DeltaIoT.v1

Figure 5.2 shows the results for online adaptation with both learning techniques and the reference approach. The results show that there is no significant difference in the results between all approaches for the two quality goals that are used to reduce the adaptation space: packet loss and latency. The same applies to the results for energy consumption.

The p-values for the different qualities based on the Wilcoxon signed-rank test for all the results are between 0.51 and 1.00. On the other hand, the time required to verify the adaptation options is significantly lower with the learning approaches compared to the reference approach (mean values: 31.80s for no learning, 14.51s for classification, and 13.98s for regression; we did not find a significant difference between classification and regression with p-value 0.98). The time for the learning approaches shown in the graph is a combination of the learning time (predicting and selecting adaptation options + online learning) and the verification time. We observed that the learning time for both approaches is a small fraction of the total time spent ($< 1\%$). It is important to note the outliers in the adaptation times for classification and regression. These outliers represent the training cycles where we utilize as much time as possible to verify adaptation options so that the learner can be trained most efficiently.

Figure 5.3 show the adaptation options for one particular cycle that the classifier and regressor respectively predicted to be of class C_3 . The figure also shows the adaptation options of the other classes that were selected for verification in this cycle. These options (denoted with green stars) are the explored options as described in algorithm 1 in section 4.3.2.

5. EVALUATION

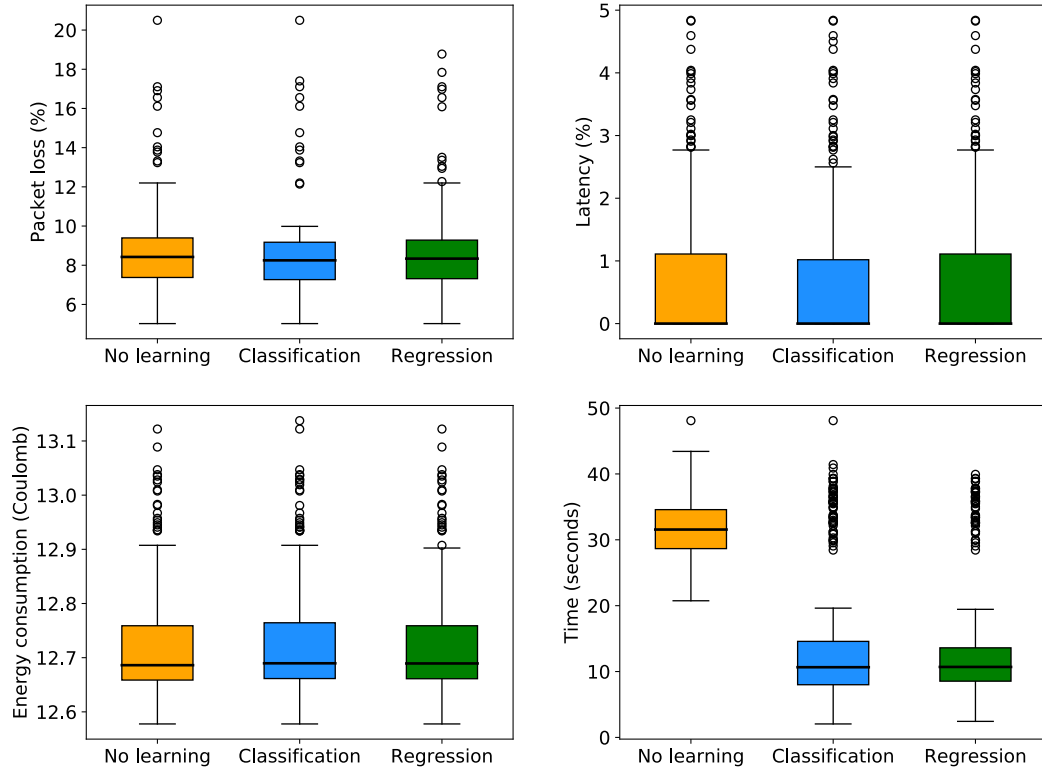


Figure 5.2: Overall results for DeltaIoT.v1

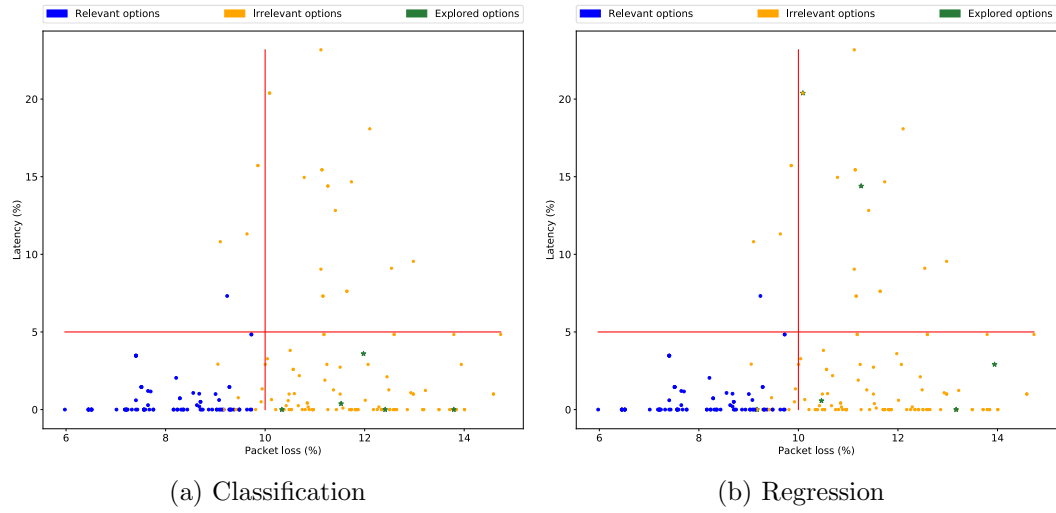


Figure 5.3: Selected and explored adaptation options in a single cycle (DeltaIoT.v1).

5.2.2 DeltaIoT.v2

Figure 5.4 shows the results for online adaptation with both learning techniques and the reference approach. It is important to note that the results for the reference approach were obtained via offline verification as exhaustive verification is not feasible within the available time slot of a cycle.

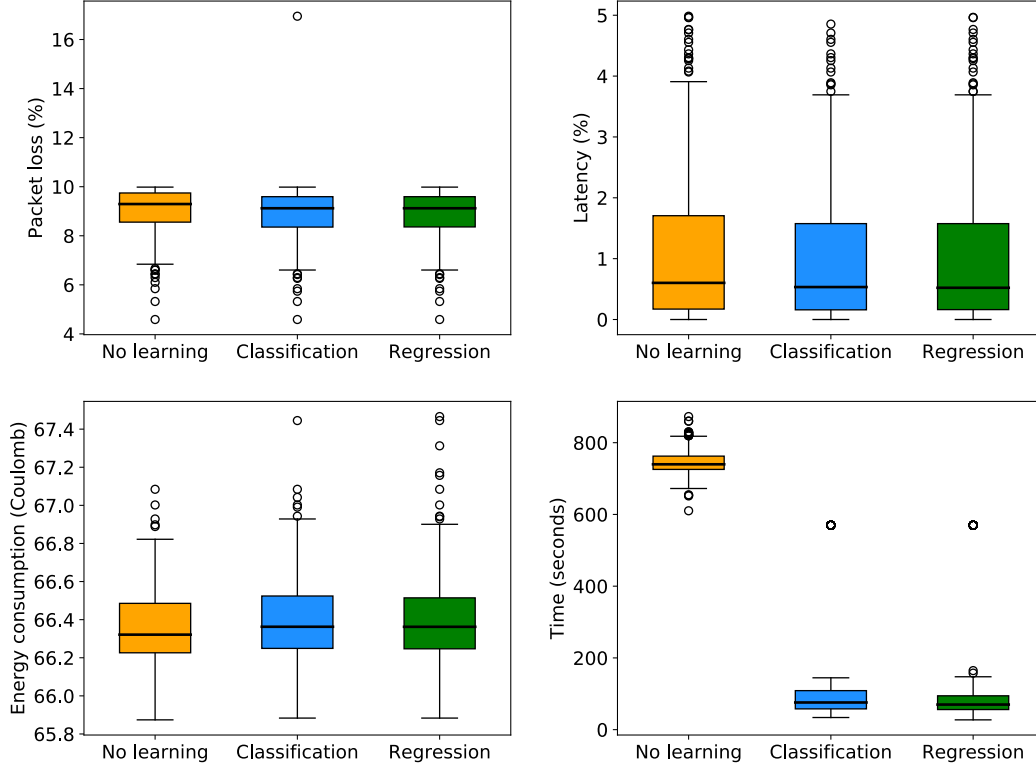


Figure 5.4: Overall results for DeltaIoT.v2

The results confirm that there is no significant difference in the results between all approaches for the three quality goals. The p-values for the different qualities based on the Wilcoxon signed-rank test for all the results are between 0.03 and 1.00. The time required for verifying the adaptation options when learning is applied is also significantly lower for DeltaIoT.v2 compared to the reference approach (mean 747.44s for no learning, 174.50s for classification, and 167.97s for regression). In this case the time for regression was significantly lower as for classification with $p = 0.0004$. The learning time remains very small compared to the verification time ($< 1\%$). The outliers for time with learning (approximately 570 seconds) also represent the time required for training here, similarly to the outliers in figure 5.2. This specific value is determined by the adaptation time that is available in the system, which is 9.5 minutes as described in section 2.2.

Figure 5.5 and 5.6 show the adaptation options for one particular cycle that the classifier and regressor selected. The legend of the figures is very similar to figure 5.3.

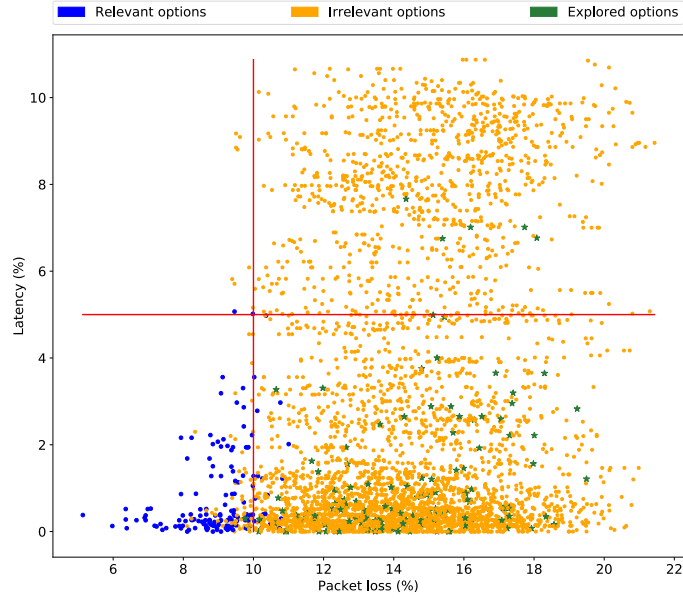


Figure 5.5: Selected and explored adaptation options in a single cycle using classification (DeltaIoT.v2)

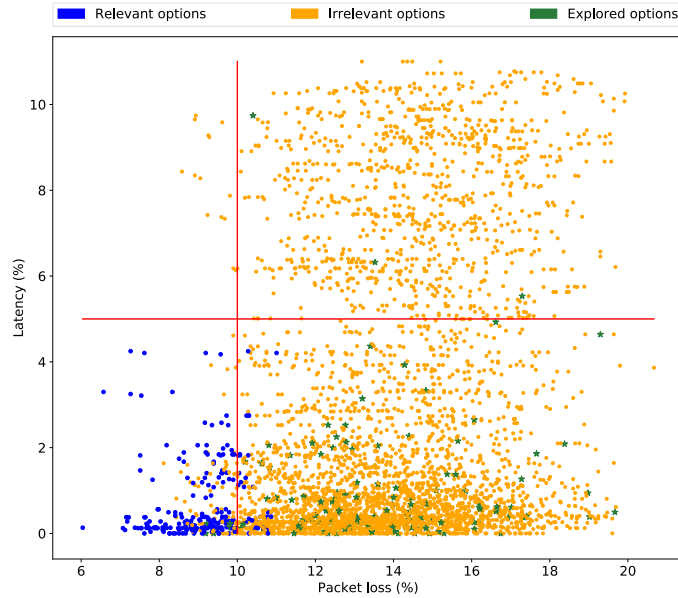


Figure 5.6: Selected and explored adaptation options in a single cycle using regression (DeltaIoT.v2)

5.2.3 Discussion

As we can see from the results retrieved from the simulation of both networks with the machine learning module, the time required for adaptation is significantly lowered. This allows us to perform a more intelligent form of analysis in DeltaIoT.v2 instead of having no way to analyze the whole adaptation space. It is therefore a definitive benefit to use the machine learning module in this case. The fact that the quality goals in the system are still retained only strengthens this conclusion. Our approach is even usable in DeltaIoT.v1, although an exhaustive approach such as ActivFORMS is preferred here since verification of the whole adaptation space is still feasible in this network.

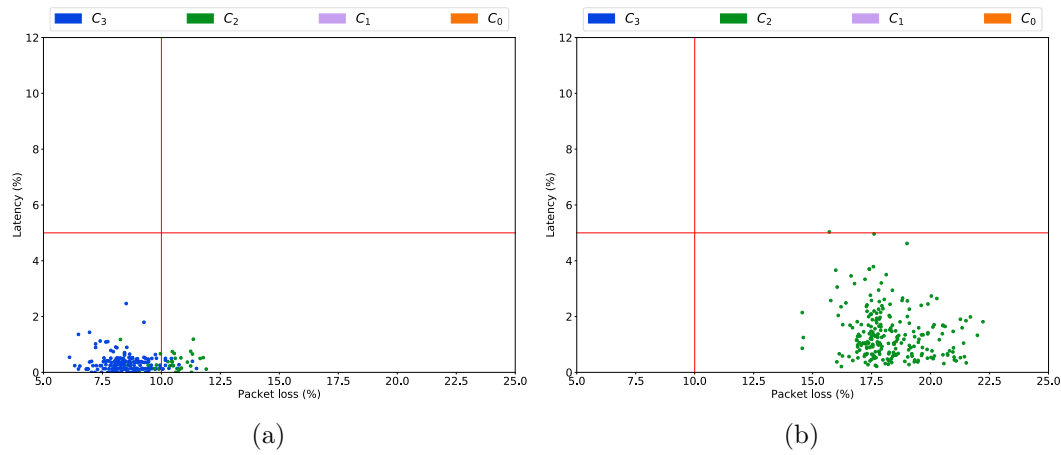


Figure 5.7: Single configurations tracked through time (DeltaIoT.v2 with classification)

Although the approach is definitely usable, it is admittedly not perfect. Configurations which mostly belong to a single class through different simulation cycles are mostly classified without problems. Figures 5.7a and 5.7b show the classification of a single configuration through 90 cycles. Configurations which cover multiple classes over time are however not always classified correctly. In some cases the configuration is still classified quite well, as can be seen in figure 5.8a. Other cases however result in the configuration being classified to a single class every cycle. This type of configuration is shown in figure 5.8b. In future work, we would like to investigate how we might be able to better handle these type of configurations.

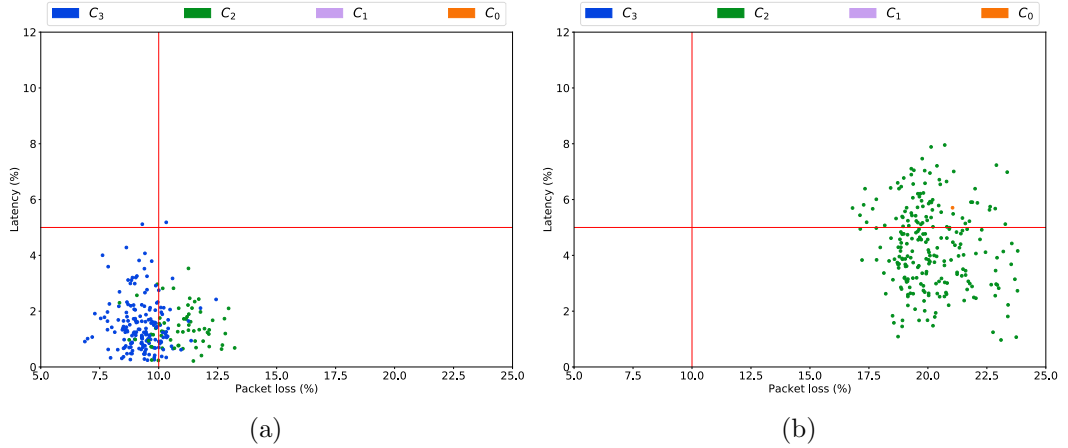


Figure 5.8: Single configurations tracked through time (DeltaIoT.v2 with classification)

5.3 Threats to validity

The evaluation of the learning approaches is subject to a number of validity threats. Since we applied the learning approach to the particular domain of IoT, we cannot generalize the conclusion (external validity). We tried to anticipate this threat to some extent by applying two different learning techniques to two different IoT networks (in terms of topology and size of adaptation space). But obviously, more extensive evaluation in different domains is required to obtain further insight.

We have evaluated the ability of the learning approach to reduce the adaptation space by measuring the impact on the quality properties of two concrete applications. The specifics of these applications, in particular the network structure, the types of uncertainties, and the specific goals that we considered may have an effect on the difficulty to demonstrate the benefits of the proposed learning approach (internal validity). We mitigated this threat to some extent by considering real application settings that are proposed by an industry partner.

The approaches are evaluated in simulation for pragmatic reasons. This simulation includes certain forms of randomness in the parameters of the setup of the applications that correspond to uncertainties. This may introduce a threat that the results may not be the same if the study would be conducted again (reliability). In order to minimize this threat, we applied two learning approaches over a period of several days that represent a large number of adaptation cycles. In addition, the complete evaluation package is available to replicate the study.

Lastly, there is a threat to validity that the probability distributions used in the simulator may not capture the real-world phenomena. More extensive testing on the real-world IoT networks outside simulation in the future can help mitigate this threat.

5.4 Conclusion

In this chapter we made an evaluation of the machine learning module by applying classification and regression techniques to the DeltaIoT networks. In the first part we went over the model selection process and the concrete instantiation of the approaches for the networks. After this initial setup, we simulated the networks and compared the results retrieved with an exhaustive approach utilizing ActivFORMS. Judging by the results we can conclude that the machine learning module made a significant difference in verification times in both networks while still maintaining the quality guarantees in the networks compared to ActivFORMS.

Chapter 6

Related work

In the recent years, machine learning has become more and more popular. This trend also applies to the world of self-adaptation. Researchers are discovering new applications of machine learning in their self-adaptive systems. This new wave of research around self-adaptation is still quite limited however. Below we will discuss related work that (1) applies machine learning in self-adaptive systems and (2) other techniques that have been studied to optimize the *analyzer* in verification of the adaptation options.

6.1 Learning in self-adaptive systems

Epifani et al. [34] present a pioneering work on learning for self-adaptive systems. The authors use probabilistic quality models that include uncertainties as model parameters. The parameters are initially set based on expert knowledge. Then, runtime data is collected and fed to a Bayesian estimator that updates the model parameters. The approach was used in [35] to support quantitative verification at runtime. Our work has a different focus, but the approach of [35] can be used in our work to keep quality models up to date.

Rodrigues et al. [36] exploit knowledge obtained from offline verification and simulations of a prototype to provide assurances for real-time adaptive systems at runtime. In addition, data mining and classification are used at runtime to learn prediction rules that are used to manage resources and decision trees to support the decision making process to conform to the verified properties in different contexts. In contrast, our approach learns the relevant adaptation options under uncertainty that are then analyzed using model checking to make adaptation decisions.

Duarte et al. [37] propose a method to learn human-readable models that explicitly capture uncertainties. These models can be used by human operators in-the-loop. The approach relies on K-plane clustering to learn possible outcomes of an action on the system quality and probabilities associated to each outcome. Expert-defined

models are used to accelerate the learning process. We apply learning to enhance the efficiency of automatic formal analysis in the feedback loop workflow.

FUSION [38] applies learning to determine the impact of adaptation decisions on the system's goals. To that end, the authors apply the M5 model tree algorithm to discover relationships between features (i.e., system capabilities) and metrics (i.e., measurable quantities, e.g., response time). As our approach, FUSION is able to improve analysis significantly, but whereas FUSION targets the feature selection space, our work targets the configuration space.

Ghahremani et al. [39] use Random Forest, Gradient Boosting Regression, and Extreme Boosting Trees to predict changes of the system utility without knowing the details of the system. The approach predicts the quality attribute values and then sorts the adaptation options based on the differences between previous and current prediction values. Our approach uses learning to select a subset of the adaptation options which are then verified by the model checker to select the optimal option.

Schmid et al. [40] make use of big data analytics tools to capture large amounts of data from the system at runtime. This data is then used in a simple model which is confined to the essential input and output parameters. This work focuses on another type of systems, i.e., large-scale software-intensive distributed systems.

6.2 Analyzer optimizations

In this section we describe different kind of approaches which deal with the increasing time and resource needs of self-adaptive systems. These approaches attempt to manage the cost of the analysis phase of the MAPE-K loop via other types of mechanisms than learning. We apply learning as a complementary approach that aims at reducing the adaptation space.

Calinescu et al. [41] apply compositional verification techniques to a large-scale complex IT system. Compositional verification splits verification tasks into smaller and more manageable sub-tasks that are responsible for verification of the system components. Assume-guarantee reasoning is often used in these type of systems. The components of the system make assumptions about the behavior of the other components, and in turn provide guarantees under these assumptions. Here we verify the assumptions made by the different components to be able to reason about the guarantees they each provide. This technique is then used to re-verify safety properties in those large-scale complex IT systems.

Johnson et al. [42] presents a framework that supports incremental verification for component-based software systems. In this framework they identify changes in the system by comparing the current state to the system state in a previous cycle, and

determine a subset of components that have to be reverified. Other components which have not changed can reuse the previous verification results.

Filieri et al. [43] uses pre-computation based verification in order to soften the workload of the analyzer at runtime. Model checking is separated into two steps: a pre-computation step and a runtime step. During pre-computation, several symbolic expressions are produced which are used to verify the compliance of system goals. These symbolic expressions are then filled in at runtime with gathered knowledge by the monitor. Verification of these expressions takes significantly less time compared to exhaustive formal model checking.

Gerasimou et al. [44] explore a combination of caching, lookahead and nearly-optimal reconfiguration techniques in order to deal with slow response times of classical formal model checking approaches. Caching performs well in systems where changes in the environment are not too frequent, allowing the reuse of previously verified results. Lookahead uses spare computation time in order to verify models that are likely to be encountered in the future. Nearly-optimal configuration stops the classic exhaustive formal verification approach prematurely when a suitable (but potentially non-optimal) configuration is found. These three approaches are evaluated on an unmanned underwater vehicle case study.

Chapter 7

Conclusion

The research question we aimed to answer in this research was:

How can machine learning techniques be used to determine subsets of adaptation options from large adaptation spaces allowing a self-adaptive system to perform more efficient analysis without compromising the realization of the adaptation goals?

We have proposed a way to extend the standard MAPE-K architecture in chapter 3. This extension is a machine learning module that is added to the *analyzer* component of the MAPE-K feedback loop. The module helps the *formal verifier* by selecting a subset of adaptation options for verification. This reduces the stress on the verifier to verify the given options within the available time.

The implementation of this module was discussed in chapter 4. Here we went over the steps necessary to configure the machine learning module (offline activities). This includes explaining how steps such as feature and model selection are conducted. Afterwards we discussed the steps that are taken at runtime (online activities).

Lastly, we have evaluated our methodology and implementation in chapter 5. We conducted the different offline activities for both networks and both techniques to obtain the machine learning module configurations. We used these configuration to evaluate the two machine learning techniques.

The results in chapter 5 demonstrate that the approach we suggested can definitely answer the research question positively. Both classification and regression techniques allowed us to select a small subset of adaptation options in the networks for further verification. The selection of this subset did not undermine the system goals while greatly reducing the adaptation time of the systems. This allows systems with larger adaptation spaces to still utilize formal verification without compromising the system requirements/goals.

7.1 Future work

Our future work is planned in three directions. First, we plan to explore learning to deal with more than two goals, include optimization goals (e.g., minimizing energy usage in the IoT cases), and dynamic goals instead of only static goals. Second, we plan to study other learning techniques to tackle the problem of adaptation space reduction, e.g. deep learning, reinforcement learning, genetic algorithms, ... Third, we plan to investigate the theoretic bounds on the guarantees for goals that can be achieved when online learning is applied to reduce the adaptation space of a self-adaptive system. [45] provides some inspiring ideas here.

We also encourage future researchers to apply the proposed techniques in this research to different domains. Since we only evaluated IoT networks, further thorough investigation in applying this approach to other type of applications is necessary to be able to generalize the results of this research.

Another interesting idea for future work is to provide a more generalized way of embedding classification techniques in software applications. This would aid future researchers/developers in specifying software systems that make use of such techniques.

7.2 Reflection

This past year has been a really great learning experience. I have learned how scientific research is conducted in practice, which seemed like something distant/faraway beforehand. I have also learned a lot about self-adaptation, formal verification and machine learning while working on this thesis. Even though this learning process wasn't always straightforward, it was always very rewarding at the end.

During this thesis I've interacted with multiple people (other students, researchers, ...). Interaction ranged from feedback/questions about the work I presented to collaboration on a part of the work. Personally, I really liked this way of working. Everyone has their own perspective and looks at the problem from a different angle. After working for quite some time on one particular problem, it is easy to get fixated on a single solution. Feedback and input from colleagues is in that regard invaluable.

Attending and presenting our paper at SEAMS has definitely been the highlight this year. Being able to present our research was truly enjoyable. Talking about it afterwards with multiple people who are quite knowledgeable in the field has been really interesting and stimulating. Other talks given during the conference were also genuinely enjoyable. It has given me quite a lot more insight into different aspects of self-adaptation, and where those aspects are situated in the field. Feedback is always well thought out and welcomed by everyone in the community. It was truly evident that each and every person that was present there is really passionate about self-adaptation and future research in this field.

Appendices

Bibliography

- [1] M. U. Iftikhar, G. S. Ramachandran, P. Bollandse, D. Weyns, and D. Hughes, “Deltaiot: A self-adaptive internet of things exemplar,” in 2017 IEEE/ACM 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, 2017.
- [2] F. Quin, D. Weyns, T. Bamelis, S. S. Buttar, and S. Michiels, “Efficient analysis of large adaptation spaces in self-adaptive systems using machine learning,” in 2019 IEEE/ACM 14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, 2019.
- [3] N. Esfahani, E. Kouroshfar, and S. Malek, “Taming uncertainty in self-adaptive software,” in 19th Symposium and the 13th European Conference on Foundations of Software Engineering, pp. 234–244, ACM, 2011.
- [4] D. Perez-Palacin and R. Mirandola, “Uncertainties in the modeling of self-adaptive systems: A taxonomy and an example of availability evaluation,” in Proceedings of the 5th ACM/SPEC International Conference on Performance Engineering, ICPE ’14, (New York, NY, USA), pp. 3–14, ACM, 2014.
- [5] S. Mahdavi-Hezavehi, P. Avgeriou, and D. Weyns, “A classification framework of uncertainty in architecture-based self-adaptive systems with multiple quality requirements,” in Managing Trade-Offs in Adaptable Software Architectures, pp. 45–77, 01 2017.
- [6] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” Computer, vol. 36, pp. 41–50, Jan. 2003.
- [7] B. H. C. Cheng, R. de Lemos, H. Giese, P. Inverardi, J. Magee, J. Andersson, B. Becker, N. Bencomo, Y. Brun, B. Cukic, G. D. M. Serugendo, S. Dustdar, A. Finkelstein, C. Gacek, K. Geihs, V. Grassi, G. Karsai, H. M. Kienle, J. Kramer, M. Litoiu, S. Malek, R. Mirandola, H. A. Müller, S. Park, M. Shaw, M. Tichy, M. Tivoli, D. Weyns, and J. Whittle, “Software engineering for self-adaptive systems: A research roadmap,” in Software Engineering for Self-Adaptive Systems [outcome of a Dagstuhl Seminar], pp. 1–26, 2009.
- [8] R. de Lemos, D. Garlan, C. Ghezzi, H. Giese, J. Andersson, M. Litoiu, B. R. Schmerl, D. Weyns, L. Baresi, N. Bencomo, Y. Brun, J. Cámara, R. Calinescu,

- M. B. Cohen, A. Gorla, V. Grassi, L. Grunske, P. Inverardi, J. Jézéquel, S. Malek, R. Mirandola, M. Mori, H. A. Müller, R. Rouvoy, C. M. F. Rubira, É. Rutten, M. Shaw, G. Tamburrelli, G. Tamura, N. M. Villegas, T. Vogel, and F. Zambonelli, “Software engineering for self-adaptive systems: Research challenges in the provision of assurances,” in Software Engineering for Self-Adaptive Systems III. Assurances - International Seminar, Dagstuhl Castle, Germany, December 15-19, 2013, Revised Selected and Invited Papers, pp. 3–30, 2013.
- [9] D. Weyns, “Software engineering for self-adaptive systems,” in Handbook of Software Engineering, Springer, 2019.
- [10] S. Dobson, S. Denazis, and A. Fernández et al., “A survey of autonomic communications,” ACM Transactions on Autonomous and Adaptive Systems, vol. 1, no. 2, pp. 223–259, 2006.
- [11] M. Salehie and L. Tahvildari, “Self-adaptive software: Landscape and research challenges,” ACM Transactions on Autonomous and Adaptive Systems, vol. 4, no. 2, pp. 14:1–14:42, 2009.
- [12] D. Weyns, U. Iftikhar, and J. Soderland, “Do External Feedback Loops Improve the Design of Self-Adaptive Systems? A Controlled Experiment,” in Software Engineering for Adaptive and Self-Managing Systems, 2013.
- [13] T. Vogel and H. Giese, “Model-driven engineering of self-adaptive software with eureka,” ACM Trans. Auton. Adapt. Syst., vol. 8, no. 4, 2014.
- [14] R. Calinescu, C. Ghezzi, M. Kwiatkowska, and R. Mirandola, “Self-adaptive software needs quantitative verification at runtime,” Commun. ACM, vol. 55, no. 9, pp. 69–77, 2012.
- [15] G. Tamura, N. M. Villegas, H. A. Müller, J. P. Sousa, B. Becker, G. Karsai, S. Mankovskii, M. Pezzè, W. Schäfer, L. Tahvildari, and K. Wong, Towards Practical Runtime Verification and Validation of Self-Adaptive Software Systems, pp. 108–132. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013.
- [16] G. A. Moreno, J. Cámara, D. Garlan, and B. Schmerl, “Proactive self-adaptation under uncertainty: A probabilistic model checking approach,” in Foundations of Software Engineering, pp. 1–12, ACM, 2015.
- [17] A. Filieri, G. Tamburrelli, and C. Ghezzi, “Supporting self-adaptation via quantitative verification and sensitivity analysis at run time,” IEEE Transactions on Software Engineering, vol. 42, pp. 75–99, Jan. 2016.
- [18] D. F. Mendonça, G. N. Rodrigues, R. Ali, V. Alves, and L. Baresi, “GODA: A goal-oriented requirements engineering framework for runtime dependability analysis,” Information & Software Technology, vol. 80, pp. 245–264, 2016.

- [19] J. Cámara, R. de Lemos, and N. L. et al., “Robustness-driven resilience evaluation of self-adaptive software systems,” IEEE Transactions on Dependable and Secure Computing, vol. 14, no. 1, pp. 50–64, 2017.
- [20] P. Arcaini, E. Riccobene, and P. Scandurra, “Formal design and verification of self-adaptive systems with decentralized control,” TAAS, vol. 11, no. 4, pp. 25:1–25:35, 2017.
- [21] R. Calinescu, D. Weyns, S. Gerasimou, M. U. Iftikhar, I. Habli, and T. Kelly, “Engineering trustworthy self-adaptive software with dynamic assurance cases,” IEEE Trans. Software Eng., vol. 44, no. 11, pp. 1039–1069, 2018.
- [22] G. S. Blair, N. Bencomo, and R. France, “Models@ run.time,” Computer, vol. 42, pp. 22 – 27, 11 2009.
- [23] M. U. Iftikhar and D. Weyns, “Activforms: Active formal models for self-adaptation,” in 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, ACM, 2014.
- [24] M. U. Iftikhar, J. Lundberg, and D. Weyns, “A model interpreter for timed automata,” in International Symposium on Leveraging Applications of Formal Methods, Verification and Validation, 2016.
- [25] A. David, K. Larsen, and A. Legay et al., “Uppaal smc tutorial,” International Journal on Software Tools for Technology Transfer, vol. 17, no. 4, pp. 397–415, 2015.
- [26] A. Legay, B. Delahaye, and S. Bensalem, “Statistical model checking: An overview,” in Runtime Verification (H. Barringer, Y. Falcone, B. Finkbeiner, K. Havelund, I. Lee, G. Pace, G. Roşu, O. Sokolsky, and N. Tillmann, eds.), (Berlin, Heidelberg), pp. 122–135, Springer, 2010.
- [27] D. Weyns, N. Bencomo, and R. Calinescu et al., “Perpetual assurances for self-adaptive systems,” in Software Engineering for Self-Adaptive Systems III. Assurances, pp. 31–63, Springer International Publishing, 2017.
- [28] S. B. Kotsiantis, “Supervised machine learning: A review of classification techniques,” in Proceedings of the 2007 Conference on Emerging Artificial Intelligence Applications in Computer Engineering: Real Word AI Systems with Applications in eHealth, HCI, Information Retrieval and Pervasive Technologies, (Amsterdam, The Netherlands), pp. 3–24, IOS Press, 2007.
- [29] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

- [30] K. Crammer, O. Dekel, J. Keshet, S. Shalev-Shwartz, and Y. Singer, “Online passive-aggressive algorithms,” J. Mach. Learn. Res., vol. 7, pp. 551–585, Dec. 2006.
- [31] F. Rosenblatt, The Perceptron, a Perceiving and Recognizing Automaton Project Para. Report: Cornell Aeronautical Laboratory, Cornell Aeronautical Laboratory, 1957.
- [32] F. Rosenblatt, “Principles of neurodynamics. perceptrons and the theory of brain mechanisms,” tech. rep., Cornell Aeronautical Lab Inc Buffalo NY, 1961.
- [33] Y. Freund and R. E. Schapire, “Large margin classification using the perceptron algorithm,” Machine Learning, vol. 37, pp. 277–296, Dec 1999.
- [34] I. Epifani, C. Ghezzi, R. Mirandola, and G. Tamburrelli, “Model evolution by run-time parameter adaptation,” in Proceedings of the 31st International Conference on Software Engineering, ICSE ’09, (Washington, DC, USA), pp. 111–121, IEEE Computer Society, 2009.
- [35] R. Calinescu, L. Grunske, M. Kwiatkowska, R. Mirandola, and G. Tamburrelli, “Dynamic qos management and optimization in service-based systems,” IEEE Transactions on Software Engineering, vol. 37, pp. 387–409, May 2011.
- [36] A. Rodrigues, R. D. Caldas, G. N. Rodrigues, T. Vogel, and P. Pelliccione, “A learning approach to enhance assurances for real-time self-adaptive systems,” arXiv preprint arXiv:1804.00994, 2018.
- [37] F. Duarte, R. Gil, P. Romano, A. Lopes, and L. Rodrigues, “Learning non-deterministic impact models for adaptation,” in Proceedings of the 13th International Conference on Software Engineering for Adaptive and Self-Managing Systems, pp. 196–205, ACM, 2018.
- [38] A. Elkhodary, N. Esfahani, and S. Malek, “Fusion: a framework for engineering self-tuning self-adaptive software systems,” in Proceedings of the eighteenth ACM SIGSOFT international symposium on Foundations of software engineering, pp. 7–16, ACM, 2010.
- [39] S. Ghahremani, C. M. Adriano, and H. Giese, “Training prediction models for rule-based self-adaptive systems,” in 2018 IEEE International Conference on Autonomic Computing (ICAC), pp. 187–192, IEEE, 2018.
- [40] S. Schmid, I. Gerostathopoulos, C. Prehofer, and T. Bures, “Self-adaptation based on big data analytics: A model problem and tool,” in Proceedings of the 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, SEAMS ’17, (Piscataway, NJ, USA), pp. 102–108, IEEE Press, 2017.

- [41] R. Calinescu, S. Kikuchi, and K. Johnson, “Compositional reverification of probabilistic safety properties for large-scale complex it systems,” in Large-Scale Complex IT Systems. Development, Operation and Management, Springer Berlin Heidelberg, 2012.
- [42] K. Johnson, R. Calinescu, and S. Kikuchi, “An incremental verification framework for component-based software systems,” in Proceedings of the 16th International ACM Sigsoft Symposium on Component-based Software Engineering, CBSE ’13, (New York, NY, USA), pp. 33–42, ACM, 2013.
- [43] A. Filieri, C. Ghezzi, and G. Tamburrelli, “Run-time efficient probabilistic model checking,” in Proceedings of the 33rd International Conference on Software Engineering, ICSE ’11, (New York, NY, USA), pp. 341–350, ACM, 2011.
- [44] S. Gerasimou, R. Calinescu, and A. Banks, “Efficient runtime quantitative verification using caching, lookahead, and nearly-optimal reconfiguration,” in Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, SEAMS 2014, (New York, NY, USA), pp. 115–124, ACM, 2014.
- [45] P. Domingos, “A few useful things to know about machine learning,” Commun. ACM, vol. 55, pp. 78–87, Oct. 2012.